ניתוח ועיצוב מערכות מידע

# תרשים תרחישי שימוש — Use Case

הרצאה 4-5

עודכן: 16.10.2026

# סדר היום

1. **יסודות UML**
2. תרחישי שימוש
3. תיעוד
4. קשרים
5. מניתוח לקוד
6. תרגול וסיכום

UML = Unified Modeling Language · שפת מידול אחידה

# מאיפה הגענו: מהדרישות למודל

Previously on SAD — דרישות (הרצאה 3)

**מה כבר יש לנו (הרצאה 3)**

- זיהוי צרכים (Wetherbe, JAD, אבטיפוס)
- ניסוח דרישות SMART
- User Story מול דרישה קלאסית
- דרישת CRUD

**לאן ממשיכים היום**

- מטקסט חופשי למודל חזותי
- תרשים תרחישי שימוש (Use Case)
- מה המערכת עושה — ולמי
- גבול: מה בפנים ומה בחוץ

**הדרישות בטקסט הן הקלט. היום נהפוך אותן למודל שאפשר לתקשר, לאמת — ולתת ל-AI.**

ניתוח דרישות

SMART = Specific, Measurable, Achievable, Relevant, Traceable · CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה · JAD = Joint Application Development · פיתוח יישומים משותף · AI = Artificial Intelligence · בינה מלאכותית

# SDLC — שלבים ובעלי תפקידים

מי הבעלים של הדרישות, ולאן נכנס תרשים תרחישי השימוש

## שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.

| 1 ייזום | 2 חקר מצב קיים | 3 ניתוח דרישות (אנחנו כאן) | 4 חקר ישימות | 5 עיצוב | 6 פיתוח | 7 בדיקות | 8 הטמעה ותפעול |
|---|---|---|---|---|---|---|---|
| ★ בעל אחריות: PM / Program Manager | ★ בעל אחריות: Business Analyst | ★ בעל אחריות: Business Analyst | ★ בעל אחריות: PM / Program Manager | ★ בעל אחריות: Designer / System Analyst | ★ בעל אחריות: Developer (FE/BE/DBA/MW) | ★ בעל אחריות: QA | ★ בעל אחריות: DevOps |
| הנהלה בכירה והנהלות הארגונים | הנהלה בכירה והנהלות הארגונים | משתמשי מערכת ישירים | הנהלה בכירה והנהלות הארגונים | UX / Creative | Full-stack Developer | Developer (FE/BE/DBA/MW) | ארגוני תמיכה (1st / 2nd / 3rd) |
| Business Analyst | משתמשי מערכת ישירים | משתמשי מערכת עקיפים | Business Analyst | Data Scientist | Reporting / BI | Full-stack Developer | QA |
| | משתמשי מערכת עקיפים | הנהלה בכירה והנהלות הארגונים | Designer / System Analyst | Developer (FE/BE/DBA/MW) | DevOps | משתמשי מערכת ישירים | משתמשי מערכת ישירים |
| | Designer / System Analyst | UX / Creative | Data Scientist | Full-stack Developer | Designer / System Analyst | Business Analyst | משתמשי מערכת עקיפים |
| | Data Scientist | Designer / System Analyst | DevOps | Reporting / BI | | | הנהלה בכירה והנהלות הארגונים |
| | Reporting / BI | | | | | | |

ניתוח דרישות

# הסצנה: מערכת הזמנת אוכל בקמפוס

הקלט הגולמי — ראיונות עם בעלי עניין

נלווה מערכת אחת לאורך כל ההרצאה: מראיונות, דרך תרשים תרחישי שימוש (use case), ועד קוד רץ בהרצאה 13.

**🎓 סטודנט/ית**

"רבע שעה בין הרצאות, אין זמן לתור. רוצה להזמין מהטלפון, לשלם, ולדעת מתי מוכן. וצמחוני — שיהיה ברור מה מתאים לי."

**🍽️ מנהל/ת קפיטריה**

"בצהריים אנחנו קורסים. רוצה לראות הכל בלוח אחד, לעדכן מה אזל, ובסוף היום מה נמכר. מי שמזמין מראש ולא בא — האוכל לפח."

**👩‍🍳 עובד/ת מטבח**

"אני רק צריך לדעת מה להכין ובאיזה סדר. מסמן 'מוכן' והסטודנט מקבל הודעה. תשלומים? לא התפקיד שלי."

**📋 עובד/ת דלפק איסוף**

"צריך לוודא שזו ההזמנה שלו וששולמה. רוצה קוד או ברקוד. ואם משהו השתבש — אני נותן החזר."

**⚠️ הקלט גולמי, מעורפל וסותר**

צרכים מעורבבים עם פתרונות, דרישות לא-פונקציונליות חבויות בתוך הטקסט, ובעיקר — סתירות בין בעלי העניין (למשל תשלום: כרטיס סטודנט מול אשראי מול מזומן). זה החומר שנברור ממנו.

**מהראיון המבולגן הזה ייגזרו השחקנים, הדרישות ותרשים תרחישי השימוש.**

ניתוח דרישות

# דרישות פונקציונליות — רשימה למעקב

דרישה טובה היא SMART · לכל דרישה מזהה (FR) ועדיפות לפי MoSCoW — מה חובה בגרסה הראשונה ומה יכול לחכות

תזכורת: דרישה פונקציונלית מתארת פעולה/שירות שאפשר לבדוק שקרה — לא "המערכת תהיה נוחה", אלא מה בדיוק היא עושה.

| מזהה · עדיפות | הדרישה (SMART) | מקור |
|---|---|---|
| FR-1 Should | המערכת תאפשר לסטודנט לסנן את התפריט לפי פרופיל תזונתי (צמחוני / טבעוני / ללא-גלוטן) ולהציג רק פריטים תואמים. | סטודנט |
| FR-2 Must | המערכת תשלח התראת "מוכן לאיסוף" תוך 10 שניות מרגע שהמטבח סימן את ההזמנה כמוכנה. | סטודנט |
| FR-3 Must | המערכת תאמת כל איסוף באמצעות קוד חד-פעמי בן 6 ספרות לפני מסירת ההזמנה. | דלפק |
| FR-4 Must | המערכת תאפשר למנהל לסמן פריט כ"אזל", ותסיר אותו מיידית מהתפריט הזמין להזמנה חדשה. | מנהל |
| FR-5 Won't | בסוף כל יום המערכת תפיק דוח מכירות המפרט כמות ומחזור לכל פריט, לפי קפיטריה. | מנהל |
| FR-6 Must | המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם. | מנהל |
| FR-7 Could | המערכת תאפשר לסטודנט לשמור הזמנה כמועדפת ולשחזר אותה להזמנה חדשה בלחיצה אחת. | סטודנט |

Must = חובה (ליבת המערכת) · Should = רצוי · Could = אפשרי · Won't = לא בגרסה זו

**המזהה והעדיפות מלווים כל דרישה — למעקב, בדיקה, ושיוך לתרחיש שמממש אותה.**

FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

ניתוח דרישות

# עד כמה מדויק — מספיק?

גם דרישה שנראית חדה מסתירה החלטה שטרם התקבלה

המנתח לא מנסח את הדרישה המושלמת בניסיון אחד — הוא יודע איזו שאלה חושפת את החסר. לכל דרישה מלמטה, השאלה שמחדדת אותה:

התראת "מוכן" תוך 10 שניות ← **ומה אם ההתראה נכשלה — יש גיבוי? מה נחשב "נמסרה"?** reliability

קוד חד-פעמי בן 6 ספרות ← **מאיפה הקוד, ומי מקליד — הסטודנט או הדלפק?** source / actor

להסיר פריט שאזל מהתפריט ← **"הסרה" = נעלם לגמרי, או מוצג כ"לא זמין"?** semantics

דוח מכירות בסוף היום ← **נוצר איפה, נשלח למי, ולמה לא לפי דרישה?** source / dest / timing

⚠️ **הדרישה לעולם לא "גמורה"**

תמיד יש שאלה שמחדדת. אותו הרגל שאתם מפעילים על פלט של AI — הפעילו על הדרישה עצמה: מה עוד לא נאמר?

**המיומנות של המנתח: לדעת איזו שאלה לשאול — לא לנסח מושלם מהניסיון הראשון.**

ניתוח דרישות

# אותן דרישות — הפעם עד הסוף

דרישה טובה סוגרת כל החלטה פתוחה — עד שאפשר לבדוק שהתקיימה

✓ מפורטת ✓ חד-משמעית ✓ ניתנת לבדיקה ✓ עקיבה (מזהה)

| מזהה · עדיפות | הדרישה — מלאה ומדויקת | מה הוכרע |
|---|---|---|
| FR-1 Should | המערכת תאפשר לסטודנט לבחור **פרופיל תזונתי אחד או יותר** (צמחוני / טבעוני / ללא-גלוטן); התפריט יציג רק פריטים שסומנו כתואמים **לכל** הפרופילים שנבחרו, והבחירה תישמר לפרופיל להזמנות הבאות. | שילוב פרופילים + שמירה |
| FR-2 Must | עם סימון ההזמנה כ"מוכנה", המערכת תודיע לסטודנט תוך 10 שניות, ו**תוודא שההודעה נמסרה** — עם **ניסיון גיבוי** אם המסירה הראשונה נכשלה. | מסירה מובטחת + גיבוי |
| FR-3 Must | בעת אישור ההזמנה המערכת תפיק קוד איסוף בן 6 ספרות **ותציג אותו לסטודנט באפליקציה**; **עובד הדלפק יקליד** את הקוד שהסטודנט מציג, וההזמנה תשוחרר רק בהתאמה. | מקור הקוד + מי מקליד |
| FR-4 Must | בסימון פריט כ"אזל", המערכת תציג אותו בתפריט כ**"לא זמין" (מעומעם, ללא הוספה לעגלה)** ולא תמחק אותו; הפריט יחזור להיות זמין בביטול הסימון. | משמעות "הסרה" |
| FR-5 Won't | בסוף כל יום עסקים (**23:00**) המערכת תפיק אוטומטית דוח מכירות לכל קפיטריה — כמות ומחזור לכל פריט — ו**תשלח אותו בדוא"ל למנהל**; הדוח יהיה זמין גם להפקה לפי דרישה. | תזמון + יעד הדוח |

**אבל דרישה מפורטת עדיין אוגדת כמה התנהגויות — כדי שכל אחת תהיה אטומית וניתנת לבדיקה, מפרקים אותה לתת-דרישות (מיד).**

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

# דרישה כללית ← דרישות מפורטות

דוגמה — ניהול זמינות פריטים (FR-4 מהרשימה, עכשיו מפורטת)

**FR-4** דרישה כללית | שחקן: מנהל | Must

**המערכת תאפשר למנהל לנהל את זמינות פריטי התפריט.**

**הדרישות המפורטות (SMART, ניתנות לבדיקה):**

**FR-4.1** המערכת תאפשר למנהל לסמן פריט תפריט כ"אזל" או להחזירו למצב "זמין".

**FR-4.2** המערכת תציג פריט "אזל" בתפריט כ"לא זמין" (מעומעם, ללא אפשרות הוספה לעגלה), ולא תמחק אותו.

**FR-4.3** המערכת תשקף שינוי זמינות לסטודנטים תוך 5 שניות מרגע העדכון.

**FR-4.4** המערכת תאפשר שינוי זמינות של פריט רק למשתמש בהרשאת "מנהל".

**הכללית אומרת מה מתאפשר; המפורטות מגדירות את ההתנהגות המדויקת.**

ניתוח דרישות

Functional Requirement = FR · דרישה פונקציונלית · Specific, Measurable, Achievable, Relevant, Traceable = SMART · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

# דרישה כללית ← דרישות מפורטות

דוגמה — אימות ומסירת הזמנה (FR-3 מהרשימה, עכשיו מפורטת)

**FR-3** דרישה כללית | שחקן: עובד דלפק | Must

**המערכת תאפשר לעובד הדלפק לאמת ולמסור הזמנה באמצעות קוד איסוף חד-פעמי.**

**הדרישות המפורטות (SMART, ניתנות לבדיקה):**

**FR-3.1** עם אישור ההזמנה, המערכת תפיק קוד איסוף חד-פעמי בן 6 ספרות ותציג אותו לסטודנט באפליקציה.

**FR-3.2** המערכת תשחרר הזמנה למסירה רק לאחר שעובד הדלפק הקליד קוד התואם במדויק לקוד ההזמנה.

**FR-3.3** אם הקוד שגוי, המערכת לא תשחרר את ההזמנה ותציג לעובד הדלפק הודעת שגיאה.

**הכללית אומרת מה מתאפשר; המפורטות הן שלבי התהליך (כולל חריג).**

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

# מה, לא איך — דרישה מתארת צורך, לא פתרון

המנתח מגדיר מה המערכת צריכה להשיג · איך לממש — החלטה של שלב העיצוב

מבחן מהיר: אם אפשר לדמיין שני מימושים שונים שמקיימים את המשפט — זו דרישה (מה). אם המשפט נועל מנגנון יחיד — נכנס אליו "איך".

| מזהה | הניסוח עם "איך" (הפתרון כבר בפנים) | הניסוח העדיף — מה בלבד | ה"איך" שייך ל־ |
|---|---|---|---|
| FR-4.2 | "...מוצג כ'לא זמין' (**מעומעם**, ללא הוספה לעגלה)" | "...מוצג כ'לא זמין' ולא ניתן להזמנה" | עיצוב / UI |
| FR-3.1 | "...תפיק **קוד חד-פעמי בן 6 ספרות** שהדלפק מקליד" | "המערכת תוודא שההזמנה נמסרת למי שהזמין" | מנגנון אימות |
| FR-2 | "...ההתראה תישלח ב-**Push ו-SMS**" | "המערכת תודיע לסטודנט באופן אמין ומיידי" | ערוץ תקשורת |

**ה"איך" לא שגוי — רק מוקדם: הוא שייך לעיצוב, ומשאיר למתכנן חופש לבחור. הדרישה נשארת יציבה גם כשהמימוש מתחלף.**

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

# דרישת CRUD קלאסית — ניהול עובדים והרשאות

יכולת חדשה בקמפוס: מי מורשה להיכנס למערכת ובאיזה תפקיד · פעולת "ניהול" = ארבע תת-דרישות, והשדות מוגדרים פעם אחת

**FR-6** **Must** שחקן: מנהל — המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם. — יכולת שאינה קיימת היום

**שדות העובד — מוגדרים פעם אחת (מילון נתונים):**

שם מלא · שם משתמש · ייחודי · תפקיד · דלפק / מטבח / מנהל · דוא"ל · פורמט תקין · קפיטריה · סטטוס · פעיל / לא-פעיל · טלפון · רשות

כל פעולה מפנה ל"שדות העובד" — לא חוזרת על הרשימה. חובה/רשות, ייחודיות ופורמט מוגדרים כאן, פעם אחת.

**הפעולות — כל תת-דרישה נכתבת כ"המערכת ת..." (The system shall) — פעולה שאפשר לבדוק:**

**C** **Create · יצירה** FR-6.1

6.1.1 המערכת תאפשר למנהל להוסיף עובד עם שדות החובה ולשייך לו תפקיד.

6.1.2 המערכת תקצה מזהה ייחודי ותדחה שם-משתמש שאינו ייחודי.

6.1.3 אם שדה חובה חסר/שגוי — המערכת לא תיצור את הרשומה ו**תציג שגיאה**.

**R** **Read · קריאה** FR-6.2

6.2.1 המערכת תאפשר למנהל לצפות בפרטי עובד.

6.2.2 המערכת תאפשר לחפש ולסנן עובדים לפי תפקיד, קפיטריה או סטטוס.

6.2.3 המערכת תציג לעובד את הפרופיל שלו בלבד.

**U** **Update · עדכון** FR-6.3

המערכת תאפשר למנהל לעדכן את פרטי העובד, תפקידו והרשאותיו (למעט המזהה). **במעבר ל"לא-פעיל" המערכת תחסום גישה ותשמור את הרשומה (מחיקה רכה).**

**D** **Delete · מחיקה** FR-6.4

המערכת תסיר רשומת עובד לצמיתות רק בתנאים מוגדרים (טעות / התיישנות / חוק). ← **מחיקה אינה פעולה אחת: בשקף הבא.**

**השדות מוגדרים פעם אחת · כל פעולה מפורקת לעומק שהיא צריכה — יצירה וקריאה דורשות כמה תת-דרישות, עדכון ומחיקה שורה אחת. לא תמיד שורה אחת.**

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית · CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה

# מחיקה — לא פעולה אחת

רוב ה"מחיקות" שאנחנו מכירים הן בעצם עדכון סטטוס · מחיקה אמיתית נדירה — ומוסדרת

**מחיקה רכה** **סימון "לא-פעיל" = Update**

שינוי סטטוס בלבד — הרשומה נשארת, הגישה נחסמת, וההיסטוריה נשמרת. זה מה שהמערכת צריכה ביום-יום (עובד עוזב → "לא-פעיל"). זו לא מחיקה.

**מחיקה אמיתית · Delete** **הסרת הנתון לצמיתות — מותרת רק כאשר:**

✋ טעות

הרשומה נוצרה בטעות ויש להסירה.

⏳ התיישנות

עברה תקופת השמירה שמוגדרת במדיניות/בחוק.

⚖️ דרישה חוקית

נושא-המידע ביקש למחוק את המידע האישי עליו.

💬 לדיון: האם עובד (או סטודנט) יכול לדרוש שנמחק את כל המידע עליו? — "הזכות להישכח" (GDPR). מה זה מחייב בדרישות המערכת?

**ברירת המחדל ברוב המערכות: מחיקה רכה. מחיקה אמיתית דורשת הרשאה, תיעוד (audit), ולעיתים מחיקה מדורגת (cascade).**

ניתוח דרישות

# User Stories — אותן דרישות, מבט המשתמש

אותה דרישה, מנקודת מבט המשתמש (User Story) והערך שהוא מקבל

כ[תפקיד], אני רוצה [יכולת], כדי [ערך]

- כסטודנט/ית, אני רוצה להזמין ולשלם מהטלפון מראש, כדי לאסוף בלי להמתין בתור בהפסקה של 15 דקות.
- כסטודנט/ית, אני רוצה לראות רק מנות התואמות לפרופיל התזונתי שלי, כדי לבחור מהר ובבטחה.
- כמנהל/ת קפיטריה, אני רוצה לראות את כל ההזמנות הפעילות בלוח אחד, כדי לנהל את עומס הצהריים.
- כעובד/ת מטבח, אני רוצה לראות תור הכנה לפי עמדה (חם / קר), כדי להכין בסדר הנכון.
- כעובד/ת דלפק, אני רוצה לאמת הזמנה בקוד ייחודי, כדי למסור לסטודנט הנכון ולמנוע טעויות.

⚠️ **המבחן של User Story טוב**
לכל אחת יש ערך אמיתי ב"כדי" — לא "כדי שהמערכת תעבוד". אם אין ערך למשתמש, זו לא דרישה אמיתית.

**מכל User Story טוב, נגזר תרחיש שימוש אחד או יותר.**

ניתוח דרישות

# דרישות לא-פונקציונליות (NFR) — כמה טוב

איכות ואילוץ שחלים על המערכת כולה — לא פעולה בודדת

### איך מבחינים? אותה מילה — קנה מידה אחר

**✅ פונקציונלי**
"שם פרטי ומשפחה של סטודנט יכולים להיות בעברית או באנגלית" — התנהגות של שדה מסוים.

**🔶 לא-פונקציונלי**
"כל קלט במערכת יכול להיות בעברית או באנגלית" — איכות רוחבית על כל המערכת.

### דוגמאות מהמערכת שלנו

- ביצועים: תהליך הזמנה מלא (מבחירה ועד אישור תשלום) יושלם תוך פחות מ-90 שניות.
- עומס שיא: תמיכה ב-500 הזמנות במקביל בין 12:00-14:00, ללא זמן תגובה מעל 2 שניות.
- אבטחה ופרטיות: אימות דרך מערכת הזיהוי של האוניברסיטה בלבד; לא יאוחסנו סיסמאות או פרטי אשראי.
- זמינות: זמינות של 99.5% בשעות הפעילות (08:00-20:00).

**⚠️ ה-NFR אינן תרחישי שימוש**
תרשים ה-use case מתאר פונקציונליות בלבד. את הלא-פונקציונליות מתעדים לצידו כאילוצים — הן משפיעות על כל תרחיש, אך אינן תרחיש בפני עצמן.

**פונקציונלי ← תרחיש בתרשים · לא-פונקציונלי ← אילוץ שמתועד לצד התרשים.**

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית

ניתוח דרישות

# UML - מה ולמה

Unified Modeling Language

**שפה חזותית סטנדרטית (תקן OMG) לתיאור, מידול ותיעוד של מערכות תוכנה - מנותקת משפת תכנות מסוימת.**

### מה זה

- שפה אחת משותפת לאנליסטים, מפתחים ובעלי עניין
- נוטציה, לא מתודולוגיה - לא אומרת לך איך לנהל פרויקט
- נולדה מתכנות מונחה-עצמים (OOD)

### למה UML?

- מסמכת החלטות ומאפשרת לאתגר אותן
- מנותקת משפת תכנות - לא ננעלת על מימוש מסוים
- בהירות משותפת על שחקנים, גבולות והתנהגות

**UML היא נוטציה משותפת - והמודל הוא המפרט שמתעד החלטות ומאפשר לאתגר אותן.**

UML = Unified Modeling Language · שפת מידול אחידה · OMG = Object Management Group · קבוצת ניהול אובייקטים

ניתוח דרישות

# UML לאורך מחזור חיי הפיתוח

כ-14 סוגי תרשימים; הקורס מתמקד ב-4 שנותנים את התמונה המלאה

**ניתוח דרישות**
מה המערכת צריכה לעשות

Activity · **4-5 Use Case**

**עיצוב**
מבנה והתנהגות המערכת

Composite Structure · Package · Communication · Object · **9 Sequence** · **8 State Machine** · **6-7 Class**

Timing · Interaction Overview

**מימוש · פריסה**
מהמודל לקוד ולתשתית

Profile · Deployment · Component

**מה נלמד — הציר המרכזי**

הכחולים = נלמדים בקורס (עם מספר ההרצאה). ארבעה תרשימים לתמונה מלאה: Use Case (דרישות) ← Class (מבנה) ← State + Sequence (התנהגות).

**UML מרוכז בדרישות ובעיצוב — לא לאורך כל ה-SDLC.**

ניתוח דרישות

# ההיסטוריה של UML

שלושת ה'אמיגוס' שאיחדו את השפה

**Grady Booch**

**OOAD**

רמה נמוכה, מיקוד בעיצוב

**Jim Rumbaugh**

**OMT**

רמה גבוהה, מיקוד בניתוח

**Ivar Jacobson**

**OOSE**

מיקוד ב-Use Cases

**איחוד תחת ה-OMG**

שלוש השיטות אוחדו ל-UML (תקן OMG, אמצע שנות ה-90). התרומה של Jacobson — תרשימי Use Case — היא בדיוק הנושא שלנו היום.

ניתוח דרישות

# למה ללמוד UML? — כי השוק מחפש את זה

## כמעט אף פעם לא שם המשרה — אבל כישור נדרש או מועדף בתפקידים מבוקשים

חפשו "UML" בלוחות הדרושים והוא חוזר שוב ושוב — כמעט תמיד לא כשם התפקיד, אלא כדרישה או יתרון בתיאור המשרה. Use Case הוא נקודת הכניסה, ושליטה ב-UML פותחת דלת לתפקידים כמו:

### 🔎 ניתוח

Business Analyst · Systems Analyst · Requirements Engineer

### 📦 מוצר

Product Manager · Product Owner

### 🏛️ עיצוב וארכיטקטורה

Solution Architect · System Architect · Software Designer

### ✅ בדיקות ואיכות

QA / Test Analyst · QA Automation Engineer

### 🔗 חפשו בעצמכם

[UML בלינקדאין (ישראל)] · [UML ב-Indeed]

### 🤖 ובעידן ה-AI

ה-AI ינסח טיוטת תרשים מהתיאור שלכם — אבל אתם מגדירים מה המערכת עושה ומה בגבול. ככל שתנסחו טוב יותר, כך הטיוטה טובה יותר — וגם תזהו מתי היא שגויה. זה מה שמבדיל אנליסט/ית מעולה מבינוני/ת.

**Use Case הוא נקודת הכניסה ל-UML — והשפה שמעסיקים מצפים שתדעו לקרוא ולכתוב.**

ניתוח דרישות

UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

# האם UML עדיין רלוונטי?

הפרובוקציה של 2026

**“ה-AI כותב את הקוד מהפרומפט.**
**בשביל מה לבזבז זמן על תרשימים?”**

**הפיתוי לדלג אמיתי:**

**AI** מייצר קוד ותרשימים בשניות — אז למה לטרוח?

לחץ זמן ודדליינים

“כבר נתקן תוך כדי”

**אבל זו טעות — ובשקף הבא נראה בדיוק למה.**

ניתוח דרישות

UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

# מודל הוא חשיבה, לא ציור

למה UML רלוונטי — ודווקא בעידן ה-AI

- מודל הוא חשיבה, לא ציור — התוצר הוא בהירות על שחקנים, גבולות והתנהגות
- המודל הוא המפרט (spec) שה-AI צריך — בינה מגבירה את מה שנתת לה, לטוב ולרע
- גבול המערכת וההיקף הם החלטה עסקית — לא משהו שבינה מחליטה במקומך
- המודל הוא משטח האימות — אי אפשר לאתגר קוד שלא הבנת

**הסמנטיקה חשובה עכשיו יותר מתמיד**

בעבר מתכנת קרא את המודל והשלים פערים בשכל ובשאלת הבהרה. היום מכונה קוראת אותו ומשחזרת בדיוק את מה שכתבת, כולל העמימות והטעויות. אי-דיוק כבר לא עולה שיחה; הוא עולה קוד שגוי, שנכתב בביטחון.

**עלות הציור צנחה לאפס. ערך האנליסט עובר מ׳לצייר׳ ל׳להחליט ולאמת׳.**

UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

# מהו מודל?

ולמה בכלל למדל — וגם מה מודל אינו

**הגדרה**

Dr. James Rumbaugh — "Modeling captures essential parts of the system"

**למה למדל?**

- הפשטה — מתמקדים בחיוני, מתעלמים מהרעש
- מאפשר לחקור תכונות ולחזות התנהגות לפני שבונים
- שפת תקשורת בין כל בעלי העניין
- ממפה מצב קיים (As-Is) מול מצב רצוי (To-Be)

⚠️ **אבל — המודל אינו המערכת ("Bubbles don't crash", Meyer)**
ה'בועות' בתרשים לא מתקמפלות ולא רצות — מודל יפה יכול להיראות נכון ולהיות שגוי. בעידן ה-AI נוצרים מודלים משכנעים-אך-שגויים בזול, ולכן חובה לאמת את המודל מול המציאות.

ניתוח דרישות

# משפחות תרשימי UML - מפת הקורס

איפה כל תרשים יושב - ומתי נלמד אותו

### Structural (מבני)

מה קיים במערכת — מבנה ודרישות

- Use Case היום
- Class Diagram

### Dynamic (התנהגותי)

איך המערכת מתנהגת לאורך זמן

- State Diagram
- Sequence Diagram

### Management (ניהולי)

תיעוד ותכנון אספקה — מעבר לתרשימים שנלמד

- חבילות, פריסה ורכיבים

| תרשים | משפחה | מתי בקורס |
|---|---|---|
| Use Case עכשיו | Structural (מבני) | היום (4-5) |
| Class Diagram | Structural (מבני) | הרצאה 6-7 |
| State Diagram | Dynamic (התנהגותי) | הרצאה 8 |
| Sequence Diagram | Dynamic (התנהגותי) | הרצאה 9 |

**איפה אנחנו עכשיו**

Use Case = שכבת הדרישות. כל השאר נבנה מעליה.

ניתוח דרישות



UML = Unified Modeling Language · שפת מידול אחידה

מכאן — בונים את התרשים

# תרשים תרחישי שימוש

# Use Case Diagram

גבול המערכת ← שחקנים ← תרחישים ← קשרים

# גבול המערכת (System Boundary)

מלבן עם שם המערכת — זו כל הסימוניה · מה נכנס פנימה זו החלטה עסקית

**מערכת הזמנת האוכל בקמפוס**

*בינתיים ריק — בכוונה*

### מה יחיה בפנים

רק תרחישי שימוש — כל מה שהמערכת מתחייבת לעשות. נצייר אותם בהמשך.

### מה עומד בחוץ

השחקנים — מי שמפעיל את המערכת. תמיד מחוץ למלבן, לעולם לא בתוכו.

### ומה שמעבר לגבול

משלוח לחדרים, רכש מספקים — לא מופיעים בתרשים בכלל. הגבול מתעד גם מה החלטנו לא לבנות.

### ⚠️ הבינה ממציאה פיצ'רים שמחוץ לגבול

AI מוסיף "בעצמו" יכולות מפתות שלא ביקשתם — משלוח, רכש מספקים — כולן ב-OUT. כש-AI ממציא משלוח, הוא חצה גבול שאתם הגדרתם. הגבול הוא שלכם.

**המלבן קובע תחום אחריות: בפנים — מה שהמערכת עושה · בחוץ — מי שמפעיל אותה · מעבר לו — מה שבמודע לא ייבנה.**

ניתוח דרישות



Artificial Intelligence = AI · בינה מלאכותית

# מערכת גדולה — כמה תרשימים, לכל אחד אזור משלו

כשהמערכת גדלה מפצלים לתת-מערכות — כל אחת תרשים use case נפרד עם גבול משלה

תרשים אחד עם עשרות תרחישים הופך לבלתי-קריא. מפצלים לפי אזורי אחריות (subsystems). לדוגמה — מערכת הזמנת האוכל שלנו מתחלקת לשלושה אזורים:

### 📱 אזור ההזמנה

באזור זה נדון ביכולות הרלוונטיות להזמנה: עיון בתפריט, הזמנה ותשלום, ומעקב אחר הזמנה.

### 🔍 מטבח ואיסוף

באזור זה נדון ביכולות הרלוונטיות למטבח ולאיסוף: תור ההכנה, סימון "מוכן", ואימות האיסוף.

### 📊 אזור הניהול

באזור זה נדון ביכולות הרלוונטיות לניהול: תפריט ומלאי, ודוחות.

**💡 במערכות אמת**

בנק או בית-חולים = עשרות תרשימי use case. שחקן יכול להופיע בכמה אזורים. עיקרון ה-KISS (Keep It Simple, Stupid · שמור על פשטות): כל תרשים ממוקד וקריא — לא הכול על דף אחד.

# Actor - שחקן

מי שיוזם תרחיש במערכת — אדם, מערכת אחרת, או זמן

שחקן = תפקיד (לא אדם ספציפי) שיוזם תרחיש במערכת — הוא זה שמתחיל.

## שחקנים אנושיים — כך מציירים



**סטודנט** · **מנהל קפיטריה** · **עובד מטבח · דלפק**

דמות מקל, השם מתחתיה — זו כל הסימוניה. תפקיד, לא אדם ספציפי.

## שחקנים לא-אנושיים — עם סטריאוטיפ




**מערכת הרישום** · **⏰ זמן**

מערכת הרישום שולחת אלינו כל לילה את עדכון הסטודנטים — היא מפעילה אותנו, ולכן שחקן «system» (ראש כתום).

הזמן מפעיל תרחישים מתוזמנים (דוח סוף-יום) — שחקן «time».

**✅ יוזם = שחקן**

מי שמתחיל את התרחיש — סטודנט, מנהל, עובד דלפק... וגם מערכת חיצונית שפונה אלינו, וגם ⏰ זמן.

**⛔ שניים שנראים כמו שחקן — ואינם**

- עקיף — רק מקבל פלט או מושפע (הנהלה שמקבלת דוח מודפס)
- מה שאנחנו קוראים לו — 💳 שער תשלום · 🔔 שירות התראות: אנחנו פונים אליהם כדי לחייב או לשלוח. זו קריאת API — הכיוון הוא שקובע.

**מבחן השחקן: מי יוזם? הוא מפעיל אותנו — או שאנחנו מפעילים אותו?**

ניתוח דרישות



UML = Unified Modeling Language · שפת מידול אחידה

# שחקן או לא? — דיון בכיתה

המבחן: מי יוזם? · התווכחו על כל מקרה — ואז לחצו להפוך אותו ולראות את התשובה

**1**

✅ שחקן

הוא מחזיק את הטרמינל — פותח את האפליקציות ומזין.

**2**

⛔ לא שחקן

אחזה קורא לו כדי לבנות — קריאה 19A. הוא שחקן אינו פותח מיזמות.

**3**

⛔ לא שחקן

רק מקליד פקס. אינו דיבור ומפעיל את הטרמינל — שחקן.

**4**

✅ שחקן

יוזם: מסיעה שהו מחזיק הרמיש — ומפעיל את הטרמינל כמסוגנס.

**5**

✅ שחקן «emit»

יוזם מפעיל הרמיטים מחזקים — משחק רק-אחוי מקבוץ בתקרת שקו.

**6**

⛔ לא שחקן

מחזיק משהוי — אינו משהמש פונקציונלי שר משקרת.

**7**

⛔ לא שחקן

דנים לקולי — אין לו שום אינטראקציה לא משקרת.

**8**

⛔ לא שחקן

אחזה שוכנים אינו — אותו כבר כמו שהר משקיפה: מכוון קונה.

**רמז: מי שמושפע, מי שרק מקבל פלט, ומה שאנחנו קוראים לו — אף אחד מהם אינו שחקן. רק מי שיוזם.**

ניתוח דרישות

# מערכת חיצונית מפעילה אותנו: שתי דרכים

הספק מאשר הזמנת חומרי גלם. מי השחקן? תלוי איך האישור מגיע אלינו.

### אין API: הספק שולח מייל לתיבה




- מייל שמגיע לתיבה לא מפעיל שום דבר במערכת. היא לא יודעת שהוא הגיע.
- לכן מישהו צריך ללכת ולבדוק: הזמן מפעיל את המערכת כל 10 דקות, והיא קוראת את התיבה ופועלת.
- היוזם הוא הזמן, ולא הספק. הספק אינו שחקן כאן.

### יש API: מערכת הספק קוראת לנו




- למערכת שלנו יש API, ומערכת הספק קוראת לו ברגע האישור.
- היא זו שמעירה אותנו, ולכן היא היוזמת: שחקן «system».
- אותו אירוע עסקי, שחקן אחר. מה שקובע הוא מי מפעיל את המערכת.

**השאלה: מה מעיר את המערכת שלנו?**

קוראים לה ישירות ← השחקן הוא המערכת הקוראת. היא צריכה ללכת ולבדוק ← השחקן הוא ⏰ הזמן.

⛔ "שליחת מייל" אינה תרחיש. זה צעד בתוך תרחיש: המערכת פונה לשירות הדואר, וזו קריאת API, בדיוק כמו שער התשלום.

**מייל נכנס ← ⏰ זמן שבודק את התיבה · קריאה ל-API שלנו ← שחקן «system» · מייל יוצא ← צעד, לא תרחיש.**

ניתוח דרישות

# Use Case — הגדרה

Ivar Jacobson — מה המערכת עושה עבור שחקן

**ההגדרה של Jacobson**

תיאור של רצף פעולות שהמערכת מבצעת, המניב תוצאה בעלת ערך נצפה לשחקן מסוים.

- 🤦 מאותחל תמיד ע"י שחקן — הוא היוזם
- 🎯 מניב תוצר בעל ערך לשחקן
- ❓ מתאר מה צריך לקרות — לא איך
- 📏 בגודל נכון — לא צעד טכני, לא תת-מערכת
- 👁️ מנקודת מבט השחקן — לא תהליך פנימי
- 🔗 מחובר לשחקנים בקשרים

**בדוגמה שלנו — שם = פועל + מושא**

הזמנת אוכל ) עיון בתפריט ) סימון הזמנה כמוכנה (

ניתוח דרישות

# תרחיש שימוש או לא? — דיון בכיתה

המבחן: יחידת ערך שלמה שמאותחלת ע"י שחקן? (לא צעד טכני, לא כל המערכת, לא דרישה לא-פונקציונלית)

1



הזמנת אוכל ותשלום

2



אימות איסוף בדלפק

3



התחברות למערכת (Login)

4



שלח מייל

5



ניהול הקפיטריה

6



זמן תגובה מתחת ל-2 שניות בעומס

**רמז: לא כל דבר שקורה במערכת הוא תרחיש — רק יחידת ערך שלמה לשחקן.**

ניתוח דרישות

# התרשים כמכלול — Use Case Diagram

גבול + שחקנים + תרחישים + קשר בסיסי — הכול יחד, בדוגמה פשוטה




ניתוח דרישות

## מה יש בתרשים?

- ✅ גבול — התיבה
- ✅ שחקנים — בחוץ
- ✅ תרחישים — בפנים
- ✅ קו association — שחקן↔תרחיש

### 💡 לכל תרחיש יש פירוט

התרשים הוא המפה — מאחורי כל תרחיש יש תיאור שלם. לחצו על תרחיש בתרשים.

### בהמשך — חלק ג'

קשרים בין תרחישים: «include», «extend», והורשה.

# מהראיון לתרחישים — 3 שלבים בהובלת האנליסט/ית

בכל שלב: האנליסט/ית העסקי/ת (Business Analyst) מחליט/ה ומאמת/ת · ה-AI מזרז

## 1 ראיונות

מנהל/ת ומחדד/ת את הראיונות — עד שכל המידע הדרוש נאסף.

מסייע בתיעוד ובארגון הנתונים שנאספו.

interview-notes.md

## 2 כתיבת דרישות

מוודא/ת שהדרישות נכונות ותואמות למה שהלקוח רוצה וצריך.

מסייע לנסח ולסווג (FR / NFR).

requirements.md — טבלה ממוספרת (FR-1...) למעקב ולשיוך לתרחיש

## 3 תרשים Use Case

מחליט/ה גבול, שחקנים, תרחישים וקשרים — ומאמת/ת מול הדרישות.

מפיק טיוטת תרשים מהדרישות.

תרשים use case

**האיכות בידי האנליסט/ית — ה-AI מזרז, לא מחליט.**

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

ניתוח דרישות

# למה לתעד מעבר לתרשים?

התרשים הוא המפה - לא הטיול

**שתי רמות תיעוד:**

## תיאור ברמה גבוהה

פסקה קצרה: מה התרחיש עושה ולמי.

מספיק כדי לדעת שהתרחיש קיים ולמי הוא שייך.

## תיאור מפורט — Flow of Events

כל הצעדים, החלופות והחריגים.

כאן חי הטיול עצמו — צעד אחר צעד, כולל מה שמשתבש.

**התיעוד הוא הקלט**

התיעוד הוא מה שמאפשר לאמת - ומה ש-AI צריך כדי לבנות נכון.

ניתוח דרישות

AI = Artificial Intelligence · בינה מלאכותית

# הזרימות — מה קורה בתוך תרחיש

זרימה אחת — ושני דברים שמסתעפים ממנה · שדות הכותרת (תנאי קדם וסיום) — בשקף משלהם, בהמשך

**ראשית (Main) — הזרימה**
רצף הצעדים הממוספר של מסלול ההצלחה. רק היא עומדת בפני עצמה.

**חלופית (Alternate) — ה-if / else שבתוך הזרימה**
וריאציה תקינה: "אם בכרטיס סטודנט... אחרת באשראי". מתפצלת בנקודת החלטה — וחוזרת לזרימה.

**חריג (Exception) — נתלה על צעד**
כשצעד נכשל: לא חלק מהזרימה — תלוי על הצעד, ובסופו תמיד ↩ חוזרים לזרימה או ⛔ מסיימים. ב-Visual Paradigm תפגשו אותו בשם Extension.

בספרות ובכלים תפגשו גם "זרימה חלופית" / "זרימת חריג" — אותם מושגים בדיוק.

## כך זה נראה — הזמנת אוכל

1. הסטודנט מרכיב את ההזמנה
2. בוחר אמצעי תשלום ומשלם

חלופית 2א: אם כרטיס סטודנט — חיוב ישיר · אחרת — אשראי. שני המסלולים תקינים וחוזרים לצעד 3.

חריג 2ב: התשלום נכשל — הצגת הסיבה ↩ חזרה לצעד 2

3. המערכת מציגה אישור וקוד איסוף
4. ההזמנה נכנסת לתור ההכנה

**ראשית = הזרימה · חלופית = if/else בתוכה · חריג = תלוי על צעד שנכשל — והמבנה הזה הוא בדיוק מה ש-AI יודע למלא ואתם יודעים לאתגר.**

ניתוח דרישות

Artificial Intelligence = AI · בינה מלאכותית

# לכתוב Flow of Events — מה זה בעצם?

לא סיפור — רצף של צעדים שאפשר לבדוק

**כל צעד:**
מי עשה מה, ומה המערכת החזירה. פעולה אחת שניתן לראות — לא סיכום עסקי.

✗ **צעד שמסתיר תת-תהליך**
"הסטודנט משלם על ההזמנה" · "המערכת מטפלת בבקשה"

✓ **אותו דבר, מפורק**
1. סטודנט: מזין מספר כרטיס, תוקף ו-CVV
2. המערכת: שולחת בקשת חיוב לשער התשלומים
3. המערכת: מקבלת אישור ופותחת את ההזמנה

⚠️ **"אם... אחרת" — כן, אבל כמבנה. לא כפרוזה בתוך צעד**
תנאי שהוסתר בפרוזה נעלם מהקורא. יש לנו if/else — בפורמט המובנה של העורך: `if / else if / end if`. ראו משמאל.

## ארבעה מבנים — לכל אחד בית אחד, ול-VP סימון משלו

| מה אני מתאר | המבנה | כך מקלידים ב-Visual Paradigm |
|---|---|---|
| פעולה שחוזרת עד שנגמר | לולאה (loop) בזרימה הראשית | `while … end while` `loop … until` |
| החלטה עסקית — כמה תוצאות תקינות | חלופית (alt) בזרימה הראשית | `if … / else if … / end if` |
| משהו נכשל — קלט שגוי, נדחה, לא נמצא | חריג (extension) תלוי בצעד שנכשל | לוח ה-Extension — נצמד לצעד: `3.a` |
| פעולה אחרת או מסלול אחר לאותה מטרה | תרחיש נפרד (scenario) | שורה חדשה ברשימת ה-Scenario |

מילים שמורות בעורך ה-Flow of Events: מקלידים בתחילת שורה — והעורך מזהה (ומוריק) אותן. בהמשך, אחד-אחד: תרחיש · חריג · תנאי קדם וסיום.

ניתוח דרישות

# מה זה תרחיש (scenario) — ומה לא?

"ניהול X" עם תרחיש אחד מעורפל = התיעוד הכי רזה שיש

### ✓ שתי סיבות מוצדקות — ורק הן

#### 1 · פעולה נפרדת בתוך CRUD

UC מסוג "ניהול X" חייב תרחיש לכל פעולה:

הוספה | צפייה | עדכון | מחיקה

בדיוק ה-CRUD שכתבנו כדרישה — עכשיו בזרימה.

#### 2 · אותה מטרה, מסלול אחר

נתיב או טריגר שונה מהותית — "רישום רגיל" מול "רישום מאוחר עם קנס": אותה מטרה, דרך אחרת להשיג אותה.

### ✗ מה שנראה כמו תרחיש — ואינו

#### תוצאת שגיאה אחרת

"התשלום נדחה" אינו תרחיש — משהו נכשל בדרך למטרה.

← זה חריג (extension) של תרחיש קיים

#### בחירה עסקית בתוך מסלול אחד

"שילם בכרטיס סטודנט או באשראי" אינו תרחיש — שתי דרכים תקינות באותו מסלול.

← זו חלופית (alt) בתוך הזרימה

**⚠ מה AI נוטה לעשות**

כותב ל-"ניהול תפריט" תרחיש אחד — "המנהל מנהל את התפריט" — ומדלג על שלוש פעולות. עכשיו יש לכם את המבחן: ניהול = תרחיש לכל פעולה.

**תרחיש = מטרה שהושגה במסלול שלם. שגיאה ← חריג · בחירה ← חלופית.**

ניתוח דרישות

CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה · AI = Artificial Intelligence · בינה מלאכותית

# כך נראה Flow of Events מלא

אותם ארבעה מבנים — בתוך תרחיש אמיתי. לחצו על תרחיש ברשימה.

### מה לחפש

- 2 — חזור על הוספת פריטים
- 4 — לפי אמצעי התשלום
- 2.2a — חריג ↩ חוזר לשלב
- מספור: נקודה = זרימה · אות = חריג

### שני תרחישים שונים

הזמנה ותשלום — תרחיש אחד עשיר.
ניהול תפריט ומלאי — ארבעה תרחישים, אחד לכל פעולת CRUD.

### אותו תיעוד — בניב של הכלי שלנו

מה שנפתח בלחיצה על תרחיש בתרשים. הניב שונה מ-VP: כאן לולאה = "חזור" (לא while), והחריגים משולבים בתוך הטקסט — ב-VP הם בלוח ה-Extension שבתחתית. מערכות שונות — אותם מושגים בדיוק.

הזמנה ותשלום | ניהול תפריט ומלאי

## הזמנה ותשלום שחקן: סטודנט

**תנאי קדם**
- הסטודנט מזוהה באפליקציה.
- לקפיטריה שנבחרה יש תפריט פעיל.

**תנאי סיום**
- ההזמנה נקלטה בסטטוס "שולמה" ושובצה לתור ההכנה.
- לסטודנט הוצגו מספר הזמנה וקוד איסוף.

**תרחישים (1)**

1 **מסלול מרכזי**

תרחיש אחד — אבל בתוכו לולאה, החלטה עסקית ושלושה חריגים.

### מסלול מרכזי

1 סטודנט: בוחר קפיטריה ופותח את התפריט שלה

2 חזור *הסטודנט מוסיף פריטים להזמנה:*

2.1 סטודנט: בוחר פריט מהתפריט ומזין כמות

2.2 *המערכת:* מוסיפה את הפריט לעגלה ומחשבת סכום ביניים

2.2a אם הפריט אזל מהמלאי בזמן הרכבת ההזמנה:

2.2a1 המערכת מסמנת את הפריט "אזל" ומסירה אותו מהעגלה

2.2a2 המערכת מציעה פריטים חלופיים מאותה קטגוריה

↩ חזרה לשלב 2.1

2.3 סטודנט: מעדכן כמות או מסיר פריט מהעגלה במידת הצורך

3 סטודנט: מאשר את העגלה ועובר לתשלום

4 לפי *אמצעי התשלום שבחר הסטודנט:*

4.1 כרטיס סטודנט

4.1.1 סטודנט: מאשר חיוב של חשבון הסטודנט שלו

4.1.2 *המערכת:* שולחת בקשת חיוב למערכת החשבונות של האוניברסיטה ומקבלת אישור

4.1.2a אם מערכת החשבונות דוחה את החיוב (יתרה לא מספיקה):

4.1.2a1 המערכת מציגה את סיבת הדחייה ומשאירה את העגלה שמורה

ניתוח דרישות

CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה

# מתי כותבים חריג (extension)?

גם חלופית וגם חריג הם "תלוי במשהו" — אז מה מבדיל ביניהם?

**הקו המפריד הוא <u>הצלחה</u> — לא ודאות**

השאלה היחידה: האם המערכת הצליחה לעשות את מה שהשחקן ביקש?

**חלופית (alt) — כן הצליחה**

השחקן בחר מסלול תקין אחר. כל ענף = מערכת עובדת.

"שילם בכרטיס סטודנט" · "שילם באשראי"
שניהם — ההזמנה שולמה.

**חריג (extension) — לא הצליחה**

המערכת לא יכלה לבצע. קלט שגוי · נדחה · לא נמצא · אזל · timeout.

"הסולק דחה את החיוב" · "קוד האיסוף שגוי"
תלוי בצעד שנכשל — לא ברשימה נפרדת.

**כל חריג חייב להיגמר — לרשום את הבעיה זה חצי עבודה**

↩ חזרה לשלב 2.1
השליטה חוזרת לצעד מסוים

⛔ המקרה מסתיים ללא הצלחה
ומציינים באיזה מצב נשארנו

**אם אינכם מוצאים אף דרך שתרחיש נכשל — עוד לא חשבתם עליו.**

ניתוח דרישות

# תנאי קדם ותנאי סיום

מה נכון לפני שמתחילים — ומה מובטח אחרי שסיימנו

### תנאי קדם (Pre-condition)

מה שהמערכת מניחה שנכון ברגע שהתרחיש מתחיל. אם זה לא מתקיים — התרחיש בכלל לא רץ.

אימות איסוף: ההזמנה מסומנת "מוכנה לאיסוף" · עובד הדלפק מחובר לעמדה.

### תנאי סיום (Post-condition)

מה שהמערכת מבטיחה שיהיה נכון אחרי סיום מוצלח. הבטחה — לא משאלה.

אימות איסוף: ההזמנה סומנה "נאספה" ונרשם מועד המסירה · היא הוסרה מרשימת הממתינות.

**⚠️ תנאי קדם ≠ בדיקה**

אם ה-UC בודק את זה ויכול להיכשל על זה — זה צעד + חריג, לא תנאי קדם.

**✓ המבחן: אפשר לראות את זה?**

מצב שהשתנה · רשומה שנשמרה · הודעה שנשלחה. "הסטודנט מרוצה" — לא נבדק.

**💡 כאן חי ה-login**

"המשתמש מחובר" הוא תנאי קדם — ולכן login אינו use case בפני עצמו.

**⚠️ וכשנכשלנו?**

תנאי הסיום מתאר את מסלול ההצלחה. מה קורה בכישלון — נכתב בחריג (⛔ "העגלה נשמרה, ההזמנה לא נוצרה").

**שני התנאים הם חוזה — ומחוזה אפשר לכתוב מקרה בדיקה.**

ניתוח דרישות

# למה אב-טיפוס (prototype)?

מסך גס מגלה תוך דקה מה שרשימת צעדים מסתירה חודש

### 1 · אף אחד לא "קורא" זרימה

תנו למנהל הקפיטריה תשעה צעדים ממוספרים — הוא יאשר. תנו לו מסך — הוא יגיד "רגע, איפה המחיר?"

### 2 · המקום הזול ביותר לטעות

לתקן שדה חסר בסקיצה = קו במחק. אחרי הפיתוח = ספרינט.

### 3 · מגלה את מה שהזרימה פספסה

כשמציירים את מסך האיסוף מתברר מיד: מה רואים כשהקוד שגוי? המסך שואל את שאלות החריגים בשבילכם.

**⚠️ אב-טיפוס ≠ עיצוב**
קופסאות אפורות. צבעים ופונטים בשלב הזה = ויכוח על ה-איך לפני שסגרנו את ה-מה.

## אימות איסוף — סקיצה

### עמדת דלפק — אימות איסוף

קוד איסוף K-58

הזמנה #4127

תשלום שולמה

| פריט | כמות |
|---|---|
| סלט יווני | 1 |
| מיץ תפוזים | 2 |

סמן כנאספה | דווח על בעיה

*...ומה מופיע כאן כשהקוד שגוי? וכשההזמנה עדיין בהכנה?*

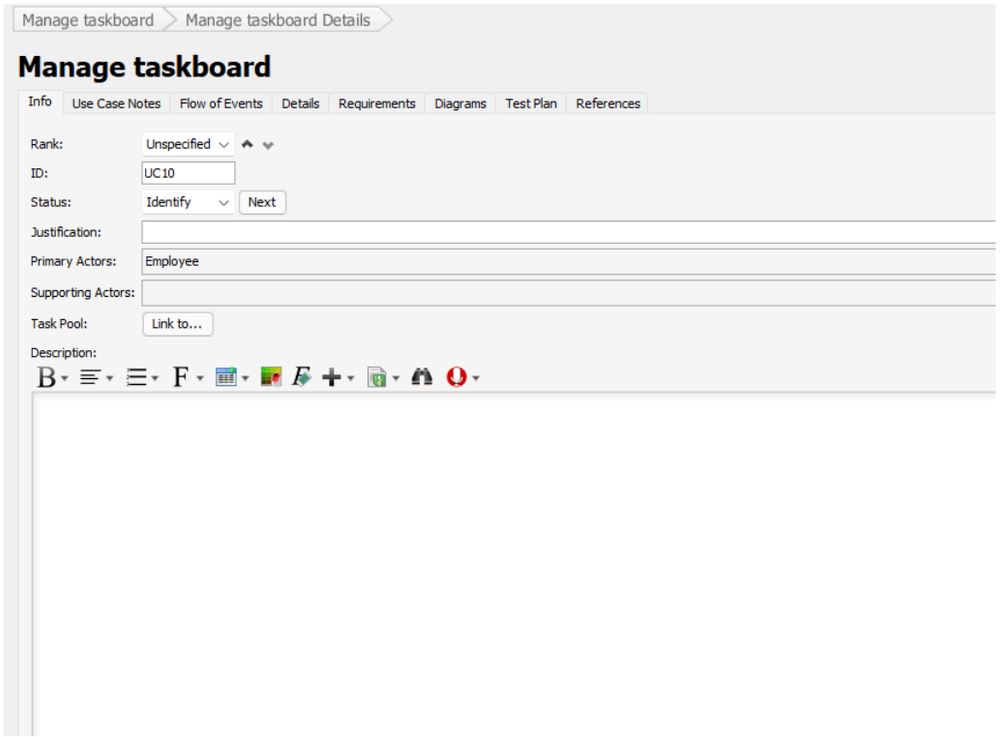
**💡 בפרויקט שלכם**
לכל use case שהשחקן שלו הוא אדם — מצרפים סקיצת מסך.

ניתוח דרישות

# איך זה נראה ב-Visual Paradigm

אותו תיעוד בדיוק — בכלי שבו תעבדו בפרויקט

Diagrams | Requirements | Details | Flow of Events | **Info**




**מי ומה.** לכל use case יש **מזהה** (UC10) ו**שחקן** — מי שיוזם. שימו לב לשפה של הכלי: VP קורא לשדה "Primary Actors" ומוסיף "Supporting Actors" — אצלנו יש פשוט **שחקנים**, וכולם יוזמים.

## לשונית = פרק בתיעוד

- Info — השחקן (VP: Primary)
- Flow of Events — תרחישים וצעדים
- Details — תנאי קדם וסיום
- Requirements — עקיבות לדרישה
- Diagrams — אב-טיפוס

### 💡 שימו לב ב-Flow of Events

התרחישים / Create / Read / Update / Delete ברשימה משמאל — CRUD[ⓘ], תרחיש לכל פעולה.

### 🤖 הכלי לא חושב במקומכם

VP נותן את הטופס. מה נכנס לתוכו — זו העבודה שלכם.

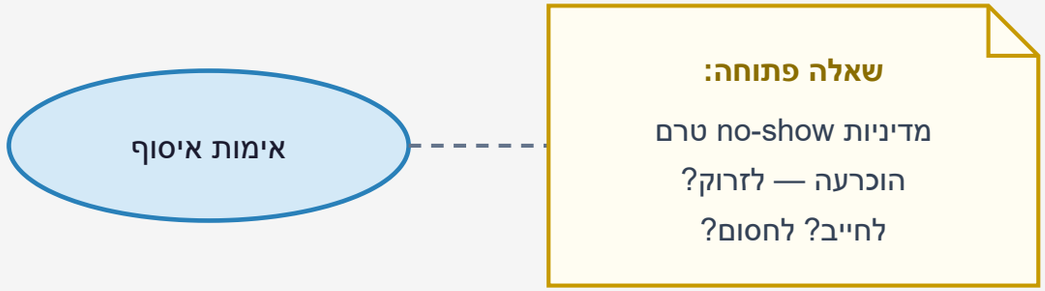
ניתוח דרישות

CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה

# מה שהתרשים לא יכול להחזיק

הנחה · משמעות של מילה · למה החלטנו — לכל אחד מהם חייב להיות בית מפורש

### הערה (Note) — הנוטציה




מלבן עם פינה מקופלת · קו מקווקו, בלי חץ · נצמדת לכל אלמנט — תרחיש, שחקן, קשר או גבול המערכת.

### ⚠️ להערה אין סמנטיקה

הכלי מתעלם ממנה. לכן היא לא דלת מילוט: מה שאפשר למדל — מדלו. הערה היא למה שבאמת לא ניתן למדל — הנחה, שאלה פתוחה, כלל עסקי.

### מילון מונחים — שם הוא החלטה, לא תווית

המלכודת אינה חוסר עקביות — היא אותה מילה, שתי משמעויות. אצלנו: "עמדה" = עמדת הכנה במטבח (חם/קר)... או עמדת הדלפק? שתי מערכות שונות.

שם התרחיש יהפוך למחלקה, לטבלה ולמסך — שינוי שם מאוחר נוגע בכל השרשרת.

### 🤖 AI "מסתדר" עם זה מצוין

הוא מתמודד "בלי בעיה" עם מילים נרדפות או דומות — ופה בדיוק הסכנה: הוא לא ייעצר לשאול "באיזו עמדה מדובר?" — פשוט יבחר אחת, בשקט.

### ניהול שינויים — לא למנוע שינוי, להוזיל אותו

שינוי אינו כישלון הניתוח. השאלה היחידה: כמה עולה לגלות מה נשבר? בשביל זה מספרנו — FR-4 השתנתה ← איזה תרחיש ממשש אותה ← איזה צעד ← איזה מסך.

שאלה פתוחה שמסומנת ככזו = יושרה. שאלה פתוחה שמתחזה להחלטה = חוב.

**המודל לא נועד להיות צודק — הוא נועד להפוך טעות לזולה.**

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית · AI = Artificial Intelligence · בינה מלאכותית

# מרשימת הדרישות לתרשים — המתכון

שישה צעדים, ולכל אחד מבחן — זה בדיוק התהליך שתעברו בפרויקט

**1** **התחילו מרשימת הדרישות**
טבלת המעקב (FR-n + שחקן-מקור + עדיפות) היא חומר הגלם — לא הראיונות הגולמיים ולא הזיכרון.

**2** **אתרו את השחקנים**
עברו על עמודת השחקן ושאלו: **מי יוזם?** אל תשכחו את הלא-אנושיים — ⏰ זמן, מערכת שדוחפת אלינו — ואת השאלה: **מי מנהל את המערכת?**

**3** **קבעו את הגבול**
מלבן, שם מערכת — ומה בחוץ נשאר בחוץ. **האם זו החלטה שלנו או של ה-AI?** הגבול הוא שלכם.

**4** **הפכו דרישות לתרחישים — לא 1:1**
על כל דרישה: **האם זו מטרה ששחקן משיג מול המערכת?** כן — תרחיש (פועל + מושא). לא — צעד בזרימה, פירוט, או אילוץ. השקף הבא עושה את זה לכל הרשימה.

**5** **חלקו לאזורים — אם צריך**
**מעל ~10 תרחישים? קבוצת שחקנים נפרדת לגמרי?** תרשים לכל אזור אחריות, כל אחד עם גבול משלו (זוכרים את שלושת האזורים?). KISS.

**6** **חברו קווים — ובדקו שלמות**
כל תרחיש מחובר לשחקן · כל שחקן עושה משהו · מטריצת ה-CRUD מכוסה · **ולמנהל המערכת יש תרחישים?** אם לא — כנראה חסר אזור שלם.

**דרישות ← שחקנים ← גבול ← תרחישים ← אזורים ← קווים ובדיקה. עכשיו נעשה את זה על הקמפוס.**

ניתוח דרישות

Functional Requirement = FR · דרישה פונקציונלית · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה · Keep It Simple, Stupid = KISS · שמור על פשטות · Artificial Intelligence = AI · בינה מלאכותית

# מדרישות לתרחישים — וזה לא מיפוי 1:1

כל הדרישות הפונקציונליות של הקמפוס — ומה עלה בגורלה של כל אחת בתרשים

| מזהה · עדיפות | הדרישה (בקצרה) | ← מה קרה לה בתרשים |
|---|---|---|
| FR-8 · Must | עיון בתפריט | ✅ תרחיש עיון בתפריט |
| FR-9 · Must | הרכבת הזמנה ותשלום | ✅ תרחיש הזמנה ותשלום |
| FR-3 · Must | אימות ומסירה בקוד איסוף | ✅ תרחיש אימות איסוף |
| FR-10 · Must | תור ההכנה במטבח | ✅ תרחיש צפייה בתור הכנה — שני יוזמים (גם המנהל) |
| FR-12 · Must | זיהוי הזמנה שלא נאספה | ✅ תרחיש טיפול בהזמנה שלא נאספה — השחקן: ⏰ זמן |
| FR-1 · Should | סינון לפי פרופיל תזונתי | 🔍 צעד בתוך תרחיש ה-alt של עיון בתפריט |
| FR-2 · Must | התראת "מוכנה לאיסוף" | 🔍 צעד בתוך תרחיש הצעד האחרון של מעקב אחר הזמנה |
| FR-4 · Must | ניהול זמינות פריטים | 🔗 שתיים ← אחד ניהול תפריט ומלאי |
| FR-11 · Must | ניהול פריטי התפריט | |
| FR-6 · Must | ניהול עובדים והרשאות | 🗺️ אזור אחר תרשים אזור הניהול — עוד שני שקפים |
| FR-5 · Won't | דוח מכירות סוף-יום | ⛔ מתועדת, לא בתרשים נדחתה במודע מגרסה 1 |
| FR-7 · Could | הזמנה מועדפת בלחיצה | ⛔ מתועדת, לא בתרשים אם יישאר זמן — תצטרף |

ושני תרחישים שלא נולדו מדרישה כללית בכלל: מעקב אחר הזמנה — מדרישה מפורטת (FR-2.5) · עדכון נתוני סטודנטים — ממילון הנתונים (ישות סטודנט 🔗, הממשק הלילי). ולעולם לא הופכות לתרחיש: NFR · חוקים עסקיים — הן חלות על התרחישים.

**⚠️ המבחן על כל דרישה**

"האם זו מטרה ששחקן משיג מול המערכת?" כן — תרחיש. לא — היא צעד, פירוט, אילוץ... או החלטה שנדחתה. זה בדיוק מה ש-AI מפספס כשהוא הופך כל שורת דרישה לבועה.

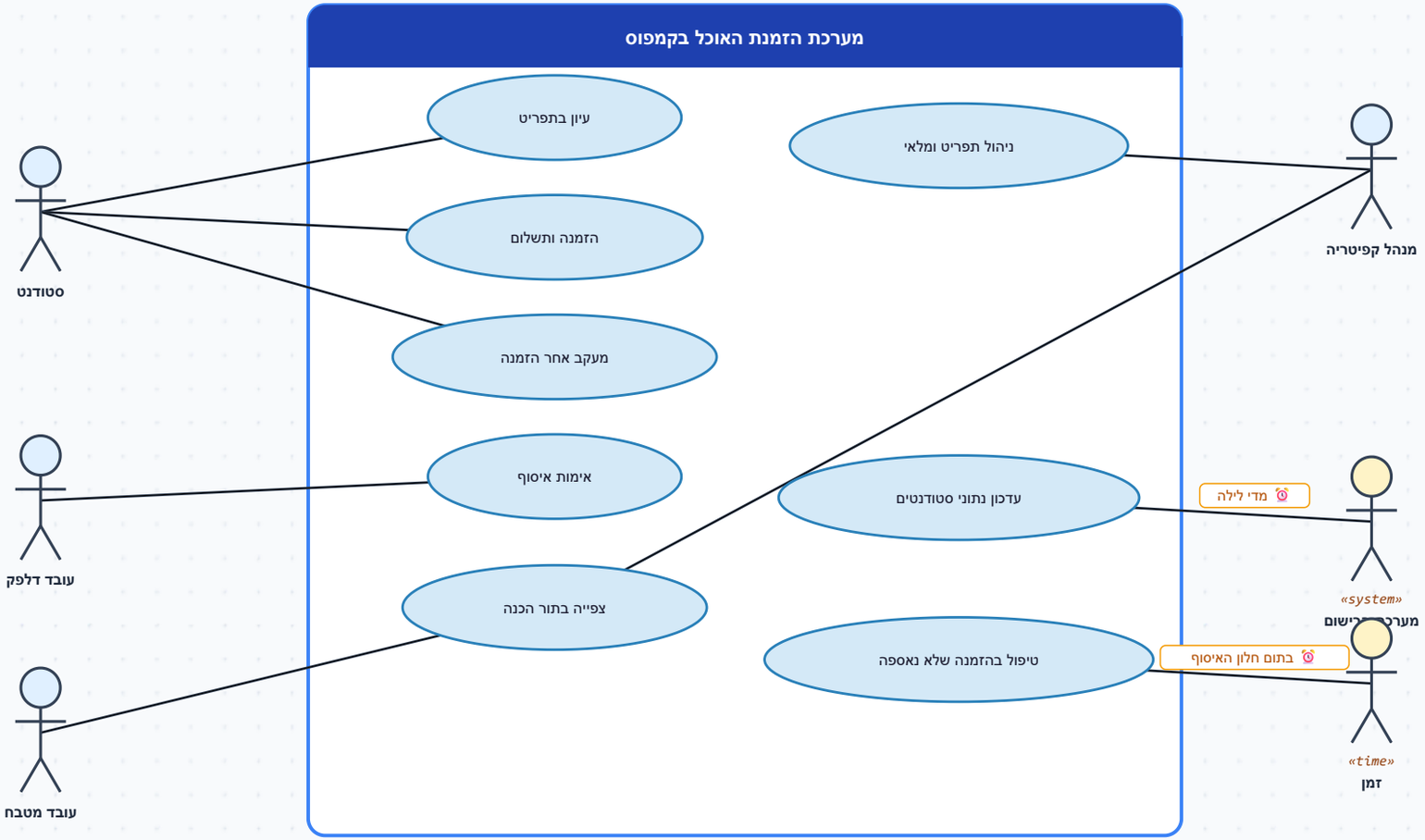
**דרישה יכולה להפוך לתרחיש, להיבלע בצעד, להתאחד עם אחרת, לעבור לתרשים אחר — או להישאר מתועדת בלבד. לא 1:1.**

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית · AI = Artificial Intelligence · בינה מלאכותית

ניתוח דרישות

# בונים את התרשים — מערכת הזמנת האוכל

גבול · שחקנים · תרחישים · קווי שיוך — בלי אף קשר "מתקדם". לחצו על תרחיש, שחקן או קו.




ניתוח דרישות

## מה לשים לב

- ⏰ זמן «time» — קו עם תווית תזמון: "בתום חלון האיסוף". חובה.
- מערכת הרישום «system» — היא דוחפת אלינו מדי לילה, ולכן שחקן.
- צפייה בתור הכנה — שני קווים: גם עובד המטבח וגם המנהל יוזמים.
- ראש כתום = שחקן לא-אנושי.

## ✅ מה שאין כאן — בכוונה

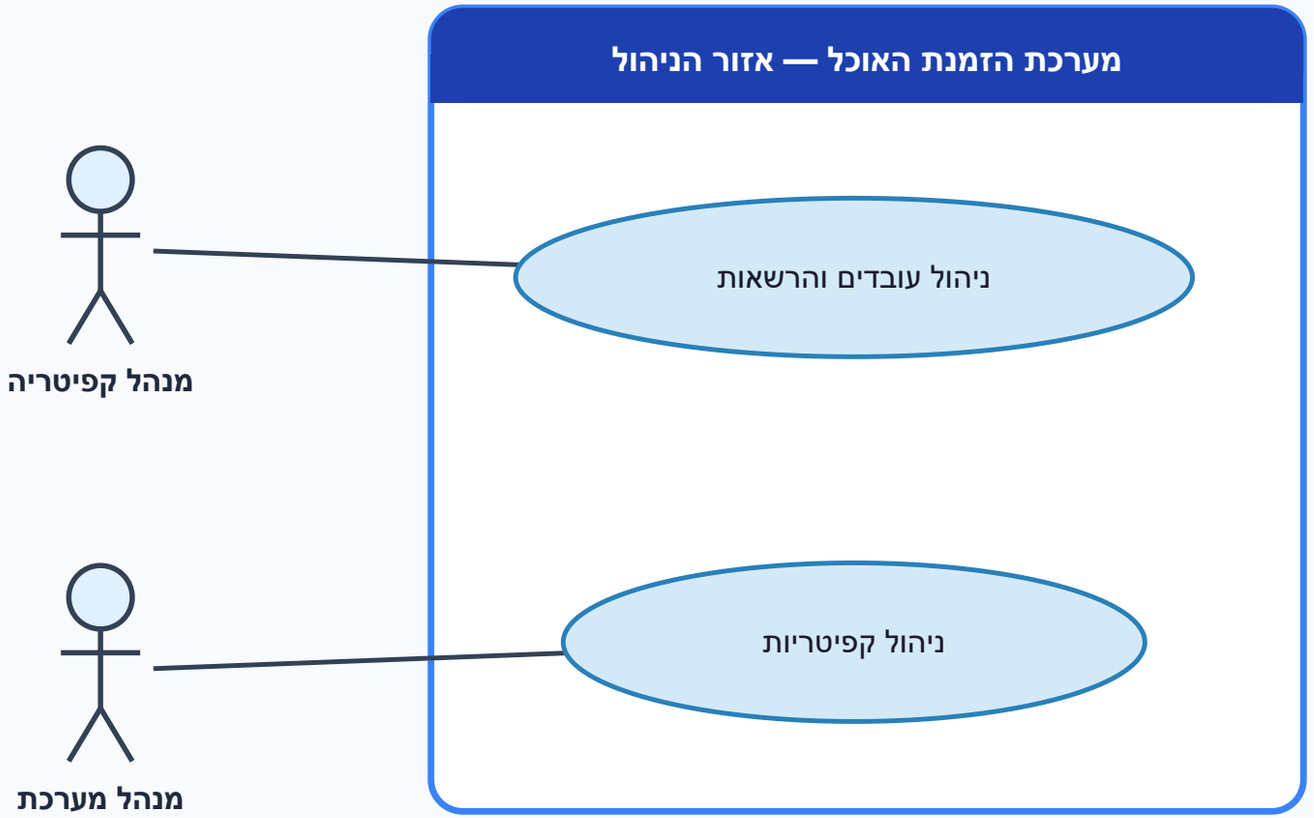
אף «include», אף «extend», אף הורשה. תרשים שלם ותקין גם בלעדיהם — הקשרים המתקדמים הם חידוד, לא תנאי.

## 🖱️ יש תיעוד מאחורי הכול

לחיצה על תרחיש פותחת את ה-Flow of Events המלא שלו — בדיוק הזרימות, החריגים והתרחישים שלמדנו.

# ומי מנהל את המערכת? — אזור הניהול

התרשים השני שהבטחנו: FR-6 לא נעלמה — היא גרה כאן




### מאיפה זה הגיע

ניהול עובדים והרשאות — FR-6 (Must).
ניהול קפיטריות — בכלל לא מדרישה: ממילון הנתונים. ישות "קפיטריה" בבעלותנו — מישהו חייב לתחזק את הרשימה.

### ✅ מנהל מערכת — כן שחקן

עבודת ניהול פונקציונלית בתוך המערכת: הגדרת קפיטריות, הרשאות. לעומת טכנאי ה-IT שמתחזק שרת (זוכרים? מקרה 6) — תשתית, לא שחקן.

### ⚠️ הכלל לפרויקט שלכם

בכל מערכת שתמדלו, שאלו: "מי מנהל את המערכת?" אם לאף שחקן אין תרחישי ניהול — כנראה חסר לכם אזור שלם. זה הצעד האחרון במתכון.

**אזור התפעול + אזור הניהול = המערכת. מי שמצייר רק את מה שהמשתמש הראשי עושה — צייר חצי מערכת.**

ניתוח דרישות



Functional Requirement = FR · דרישה פונקציונלית

# ארבעת הקשרים — וזה הכל

Generalization · extend · include · Association

## הקשרים בתרשים

**Association**
קו בין שחקן לתרחיש (תקשורת)

**«include»**
שימוש חוזר, תמיד מתבצע

**«extend»**
התנהגות מותנית, לפעמים

**Generalization**
הורשה (בין שחקנים)

**⚠️ מה זה לא קשר**

- אין סדר/רצף בין תרחישים (זה לא flowchart)
- אין «uses»/«precedes» (מ-UML 1, מת)
- אין קשר בין שני שחקנים מלבד הורשה

**אין קשר חמישי. אם צריך לתאר סדר פעולות — זה ה-flow of events, לא התרשים.**

ניתוח דרישות

# Association — הקשר הבסיסי

הקו שמחבר שחקן לתרחיש

**קו פשוט בין שחקן לתרחיש = תקשורת. זהו הקשר הבסיסי ביותר.**

### הסימון

- קו מלא — בלי חץ
- כל תרחיש מחובר לפחות לשחקן אחד
- שחקן = מי שיוזם. אין "ראשי" ואין "משני" — יש שחקנים
- לתרחיש יכולים להיות כמה שחקנים — כל אחד יכול ליזום אותו בעצמו

### דוגמה מהקמפוס




שני שחקנים לאותו תרחיש — כל אחד פותח את התור בעצמו, מתי שהוא צריך. אין ביניהם היררכיה.

**⚠️ מה שאנחנו קוראים לו — אינו שחקן**

שער תשלום שהמערכת פונה אליו הוא קריאת API — עיצוב, לא תרשים. המבחן: הוא מפעיל אותנו, או אנחנו אותו? אילו היה מתריע לנו מיוזמתו — היה שחקן.

ניתוח דרישות



UML = Unified Modeling Language · שפת מידול אחידה

# «include» — שימוש חוזר

מחלצים תת-התנהגות משותפת

- כשכמה תרחישים חולקים תת-התנהגות — מחלצים אותה לתרחיש משלה
- התרחיש המשותף תמיד מתבצע (mandatory) — ללא תנאי
- החץ: חץ מקווקו מהתרחיש הבסיסי אל המשותף, עם המילה «include»

**בדוגמה שלנו:**

**הזמנת אוכל «include» בדיקת עגלה · תשלום על הזמנה «include» בדיקת עגלה**

**הבינה מאתרת שימוש חוזר**

Claude קורא את כל התרחישים, מזהה תת-התנהגות שחוזרת בכמה מהם (למשל בדיקת עגלה — גם בהזמנה וגם בתשלום) ומציע לחלץ אותה לתרחיש «include» אחד.

**⚠️ בדקו את הכיוון והתנאי**

החץ מצביע מהתרחיש הבסיסי אל המשותף — לא הפוך. ושימו לב: «include» תמיד רץ. אם ההתנהגות מתרחשת רק לפעמים — זה כבר לא include.

**טיפ: אם תת-התנהגות חוזרת בכמה תרחישים — זה מועמד ל-include.**

ניתוח דרישות

# «include» — איך זה נראה באמת

התרשים אומר שיש שימוש חוזר — הזרימה אומרת איפה הוא נקרא

### זרימה: הזמנת אוכל

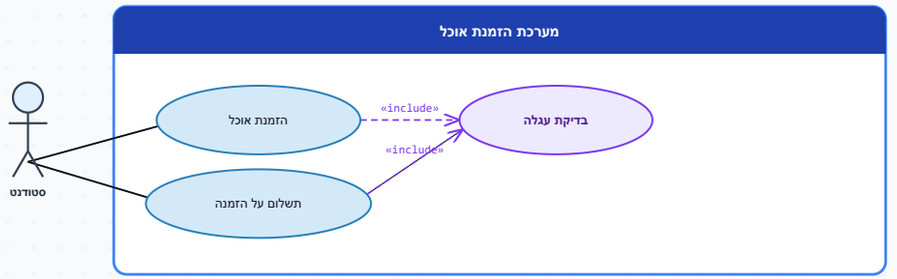
1. סטודנט: פותח את העגלה ובוחר ״בצע הזמנה״
2. **«include» בדיקת עגלה**
3. סטודנט: בוחר זמן איסוף
4. המערכת: יוצרת הזמנה ומציגה סיכום

### זרימה: תשלום על הזמנה

1. סטודנט: בוחר הזמנה ולוחץ ״שלם״
2. **«include» בדיקת עגלה**
3. סטודנט: מזין את פרטי התשלום
4. המערכת: מעדכנת את ההזמנה ל״שולמה״

### 🔗 אותה שורה — בשתי הזרימות

כל חץ מקווקו בתרשים = שורה אחת בזרימה. שם, ורק שם, ה-«include» באמת נקרא.




👆 לחצו על בדיקת עגלה — אין לה שחקן, אין לה מסך, והצעדים שלה לא יודעים מי קרא להם.

**שני בסיסים · בלי שחקן משלה · רצה תמיד — רק אז זה «include».**

ניתוח דרישות

# «extend» - התנהגות מותנית

מוסיפים התנהגות אופציונלית בתנאי

- תרחיש מרחיב מוסיף התנהגות אופציונלית לתרחיש בסיסי
- בנקודת הרחבה (extension point) ובתנאי (condition)
- החץ הפוך מ-include: מהמרחיב אל הבסיסי. קורה רק לפעמים

**הסימון: «extend»**

**Extension - - - ▷ Base**

חץ מקווקו עם ראש פתוח, מהתרחיש המרחיב אל הבסיסי — ומעליו התווית «extend».

**נקודת הרחבה ותנאי**

- נקודת הרחבה (extension point) — היכן בבסיסי נכנס המרחיב
- תנאי (condition) — מתי בכלל הוא קורה

**בדוגמה שלנו:**

**הזמנה מראש «extend» הזמנת אוכל [תנאי: איסוף עתידי]**

⚠️ **כלל אצבע — בלבול נפוץ (גם של AI)**

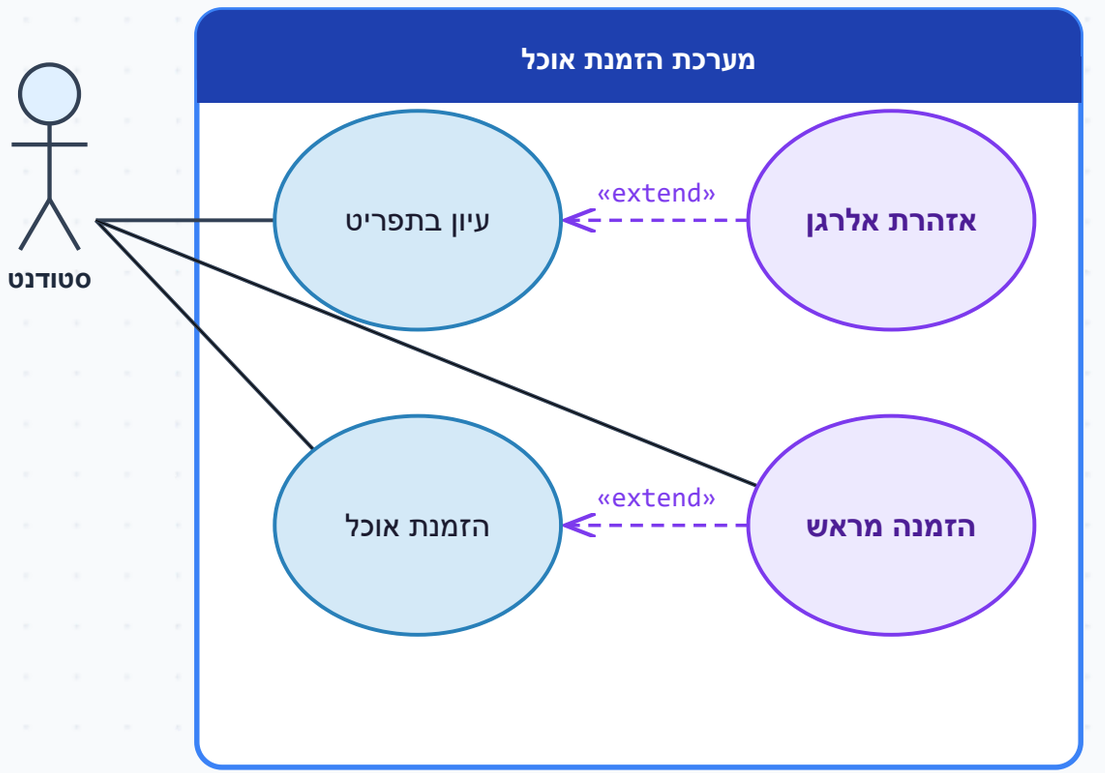
include = תמיד · extend = בתנאי. בלבול ביניהם הוא טעות נפוצה (גם של AI): מודל יסמן לעיתים התנהגות אופציונלית כ-include חובה, או הפוך. בדקו את כיוון החץ ואת התנאי.

**include = שימוש חוזר מחויב · extend = הרחבה אופציונלית בתנאי, בנקודת הרחבה.**

ניתוח דרישות

# «extend» — ולמה לא פשוט תרחיש?

אותו מה, איך אחר — והשאלה האמיתית היא מי צריך לראות את זה




### 🔍 למה לא תרחיש?

תרחיש מסתתר בתוך התיעוד. «extend» נמצא בתרשים — ורק שם הלקוח יכול להצביע ולשאול "איפה זה אצלכם?"

### 1 · הזמנה מראש — זה שאפשר להתווכח עליו

יכלה להיות תרחיש נסתר ("אותה מטרה, מסלול אחר"). בחרנו להראות — כי המנהל מודאג מ-no-show, ועכשיו הוא רואה את זה ושואל: "ומה אם הוא לא בא?"

נקודת הרחבה: קביעת זמן איסוף · תנאי: איסוף עתידי · נדיר: "לפעמים"

### 2 · אזהרת אלרגן — זה שאין עליו ויכוח

לא תרחיש: אותו מסלול בדיוק, אותה מטרה — רק התנהגות נוספת לתת-קבוצה נדירה. ובטיחותי, ולכן הלקוח חייב לראות שהמערכת תומכת בזה.

נקודת הרחבה: הצגת פרטי הפריט · תנאי: מוגדר פרופיל אלרגיה

### ⚠️ ההבדל מ-«include»

את ה-«include» קוראים — יש לו שורה בזרימה. את ה-«extend» אף אחד לא קורא: הבסיס מציין נקודת הרחבה, וההרחבה נכנסת שם — רק אם התנאי מתקיים.

👆 לחצו על ההרחבות. שימו לב: שני הבסיסים רצים מצוין בלעדיהן.

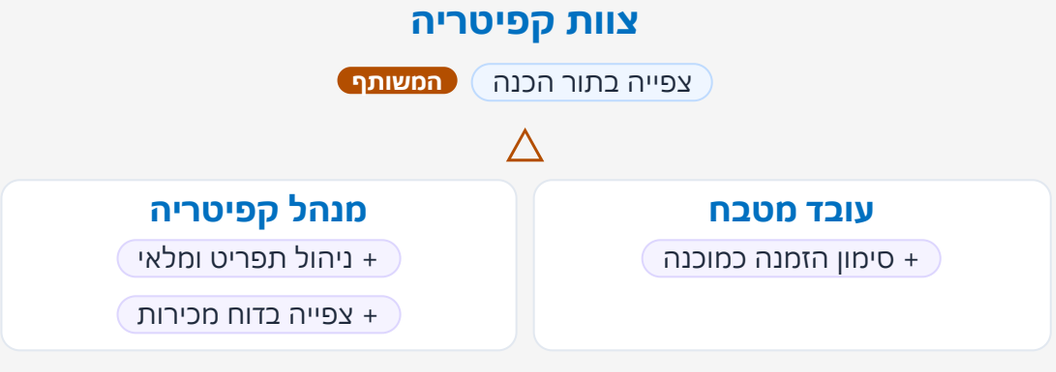
**אותו מה, איך אחר · נדיר · ורוצים שיראו אותו — רק אז «extend».**

ניתוח דרישות

# Generalization — הורשה

בין שחקנים — שחקן כללי שממנו יורשים שחקנים מתמחים

## הורשת שחקן (Actor)




עובד דלפק איסוף — לא בהיררכיה. הוא לא צופה בתור ההכנה, אז אין לו חלק במשותף (הוא עושה רק אימות איסוף). ילד שלא עושה את המשותף — לא ילד.

המשולש החלול מצביע אל ההורה — אותו סימון כמו ב-Class Diagram, כאן בין שחקנים.

## מתי זה נכון? — שני תנאים, שניהם

1. המנהל צופה בתור ההכנה בתור מנהל — זה חלק מהתפקיד שלו, **לא כממלא מקום**
2. ויש לו גם תרחישים משלו — ניהול תפריט, דוח מכירות. אחרת הוא פשוט "צוות קפיטריה", ולא צריך שחקן נפרד בכלל

נפל תנאי 1 ← ממלא מקום (למטה). נפל תנאי 2 ← הוא פשוט השחקן הכללי.
השחקן הכללי מחזיק את המשותף; כל מתמחה יורש אותו ומוסיף את שלו — וחוסך קווי association חוזרים.

## 🎭 שחקן = תפקיד, לא אדם

המנהל עומד בדלפק כי עובד הדלפק יצא להפסקה. באותו רגע הוא לא פועל בתור מנהל — הוא ממלא מקום: אותו אדם, תפקיד אחר. זו לא הורשה — הקשר שייך לעובד דלפק, והאדם פשוט חובש שני כובעים בזמנים שונים. השאלה אף פעם אינה "מי האדם?" אלא "באיזה תפקיד הוא פועל עכשיו?"

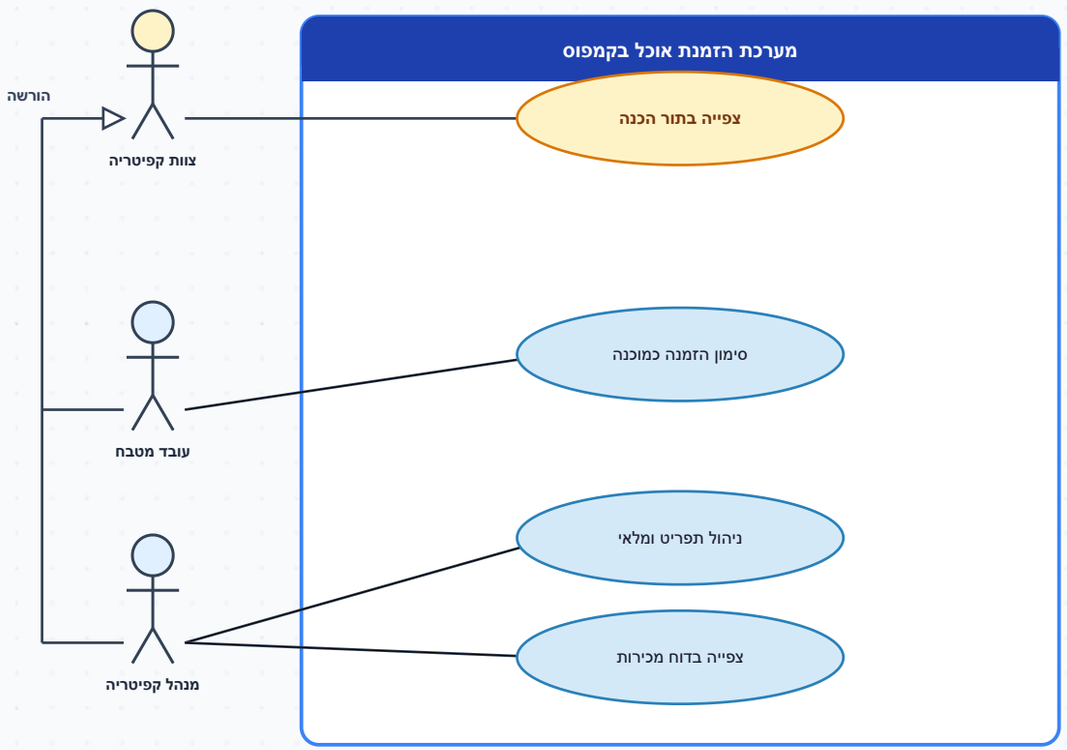
### ⚠️ בקורס זה: הורשה רק בין שחקנים

UML מתיר generalization גם בין תרחישים — אנחנו לא משתמשים בזה בקורס (תיתקלו בזה, ולא — זו לא טעות). לשיתוף התנהגות בין תרחישים: «include» (תמיד) או «extend» (בתנאי).

ניתוח דרישות

# הורשה — איך זה נראה באמת

קו אחד על ההורה — ושני התפקידים יורשים אותו




### 🔍 מה לחפש — מה שאיננו שם

אין קו מעובד מטבח או ממנהל אל "צפייה בתור הכנה". הם יורשים אותה מצוות קפיטריה. ההיעדר הזה — הוא הסימון שעובד.

### ✅ שני התנאים מתקיימים — ולכן זו הורשה

1. שניהם צופים בתור ההכנה בתור עצמם — זה חלק מהתפקיד, לא מילוי מקום.
2. ולכל אחד יש גם משלו: מטבח מסמן הזמנה כמוכנה; המנהל מנהל תפריט ורואה דוח.

נפל תנאי 2? אין ילד — הוא פשוט "צוות קפיטריה".

### 🚪 מי לא בהיררכיה — ולמה

עובד דלפק איסוף. הוא לא צופה בתור ההכנה — אין לו חלק במשותף, אז אין מה לרשת. ילד שלא עושה את המשותף — לא ילד.

ומה עם המנהל שמחליף אותו בהפסקה? זה מילוי מקום — ברגע ההוא הוא עובד דלפק.

### ⚠️ ורגע — בתרשים הגדול יש שני קווים!

נכון, ושניהם תקינים. עם תרחיש משותף אחד — שני קווים פשוטים יותר. ההורשה משתלמת כשהמשותף גדל. זו החלטה, לא כלל.

👆 לחצו על השחקנים, על התרחישים — ועל קו ההורשה עצמו.

**ההורה מחזיק את המשותף · כל ילד מוסיף את שלו · מי שלא חולק — לא ילד.**

ניתוח דרישות

# איזה קשר — אם בכלל?

לחצו על מקרה — ותקבלו שתי דרכים למדל אותו. איזו נכונה יותר?

1



בדיקת עגלה — נדרשת גם בהזמנת אוכל וגם בתשלום, בכל פעם

2



הזמנה מראש לשעת איסוף עתידית — לפעמים

3



התחברות לפני שמזמינים

4



מנהל ועובד מטבח — שניהם צופים בתור ההכנה, ולכל אחד יש גם משלו

5



תשלום — בכרטיס סטודנט או באשראי

6



הדפסת מדבקה אחרי סימון "מוכנה" — בכל פעם

7



פריט אזל בזמן ההזמנה — המערכת מציעה חלופה

8



הסטודנט תמיד מעיין בתפריט לפני שהוא מזמין

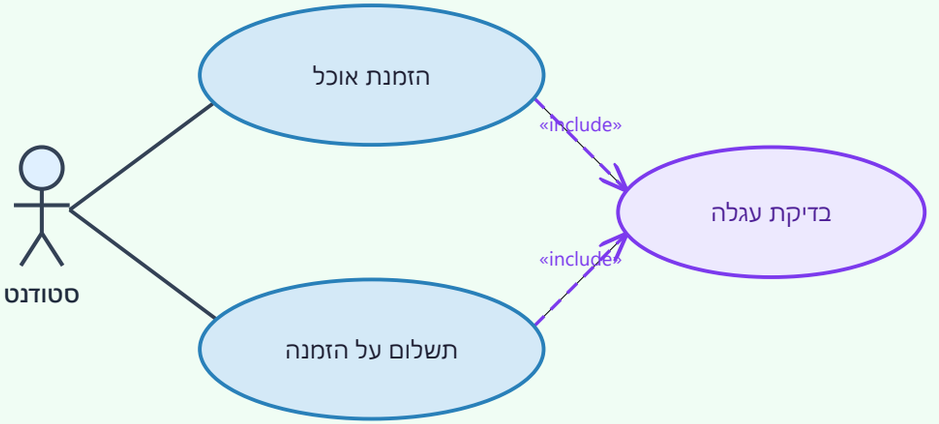
**רמז: ברירת המחדל היא אפס «include» ואפס «extend».**

ניתוח דרישות

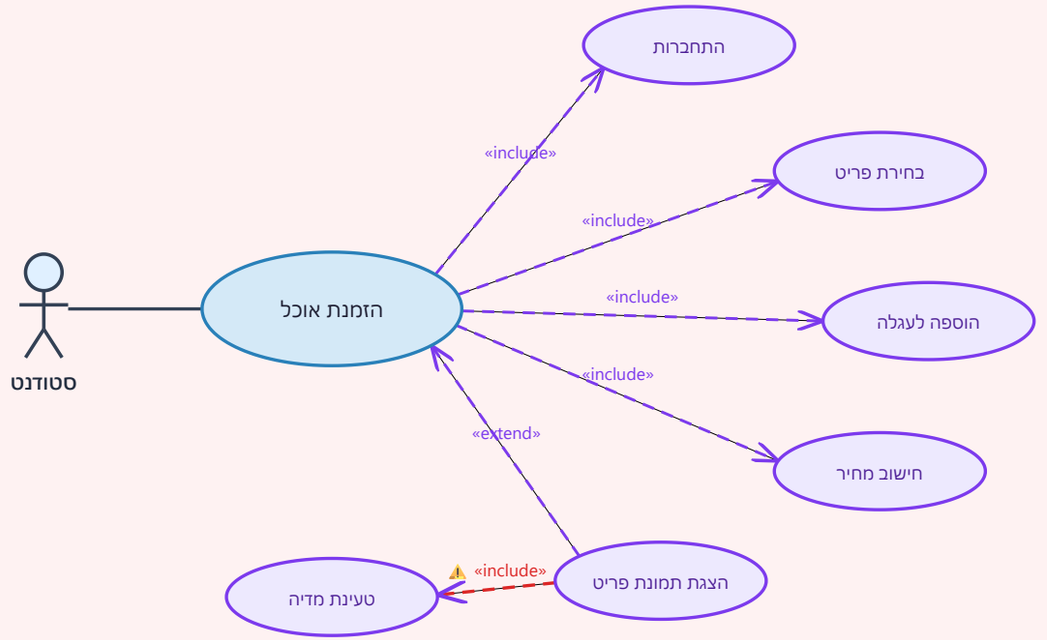
# KISS — פירוק טוב מול פירוק-יתר

אותו תרחיש: כמה תרחישים ברורים — או עשרות זעירים?

**פירוק טוב** — תרחישים שמניבים ערך, «include» אחד מוצדק




**פירוק-יתר** — כל צעד זעיר הפך לתרחיש (אנטי-תבנית)







**1 · אין**

«extend» → «include»

לא משרשרים הרחבה לתוך הכללה (הקו האדום).

**2 · «extend» → «include»**

קורה, אבל
נדיר מאוד
— עצרו ובדקו.

**3 · «extend» במשורה**

ברירת המחדל
אפס
. רוב הפריטים — צעד בזרימה.

**פשוט מנצח מפורט — התרשים מתקשר היקף וערך, לא מפרק טכני.**

ניתוח דרישות

Keep It Simple, Stupid = KISS · שמור על פשטות

# רמת פירוט ו-FAQ

כמה זה יותר מדי — וכמה מעט מדי

**לא גבוה מדי, ולא נמוך מדי**

לא כל לחיצה היא תרחיש, ולא הכול תרחיש אחד ענק. תרחיש = יחידת ערך אחת לשחקן.

### האם login הוא use case?

לא — זה תנאי קדם, לא תרחיש. וגם לא יעד «include»: אי אפשר לכלול משהו שאינו תרחיש.

### לחבר שחקן ל-«include»?

בדרך כלל לא — ליעד «include» טהור אין שחקן. חוקית מותר, אבל אם יש לו שחקן — כנראה שהוא לא באמת «include».

### לחבר שחקן ל-«extend»?

כן. להרחבה יכול להיות שחקן (הוא בוחר בה) — או לא (המערכת מפעילה אותה לפי תנאי).

### שחקן "זמן" נפרד לכל פעולה מתוזמנת?

לא — שחקן ⏰ אחד, וכמה חיצים ממנו בתדירויות שונות (יומי, שעתי, בתום חלון...).

### קשר «extend» מול "הרחבות" בטבלת ה-Flow of Events?

קשר בתרשים = תהליך שונה משמעותית. חריג בתוך הזרימה = משהו פשוט כמו if...else.

### כמה תרחישים צריך?

תלוי בהיקף — אין מספר קסם. מודדים לפי מטרות הערך של השחקנים, לא לפי מסכים.

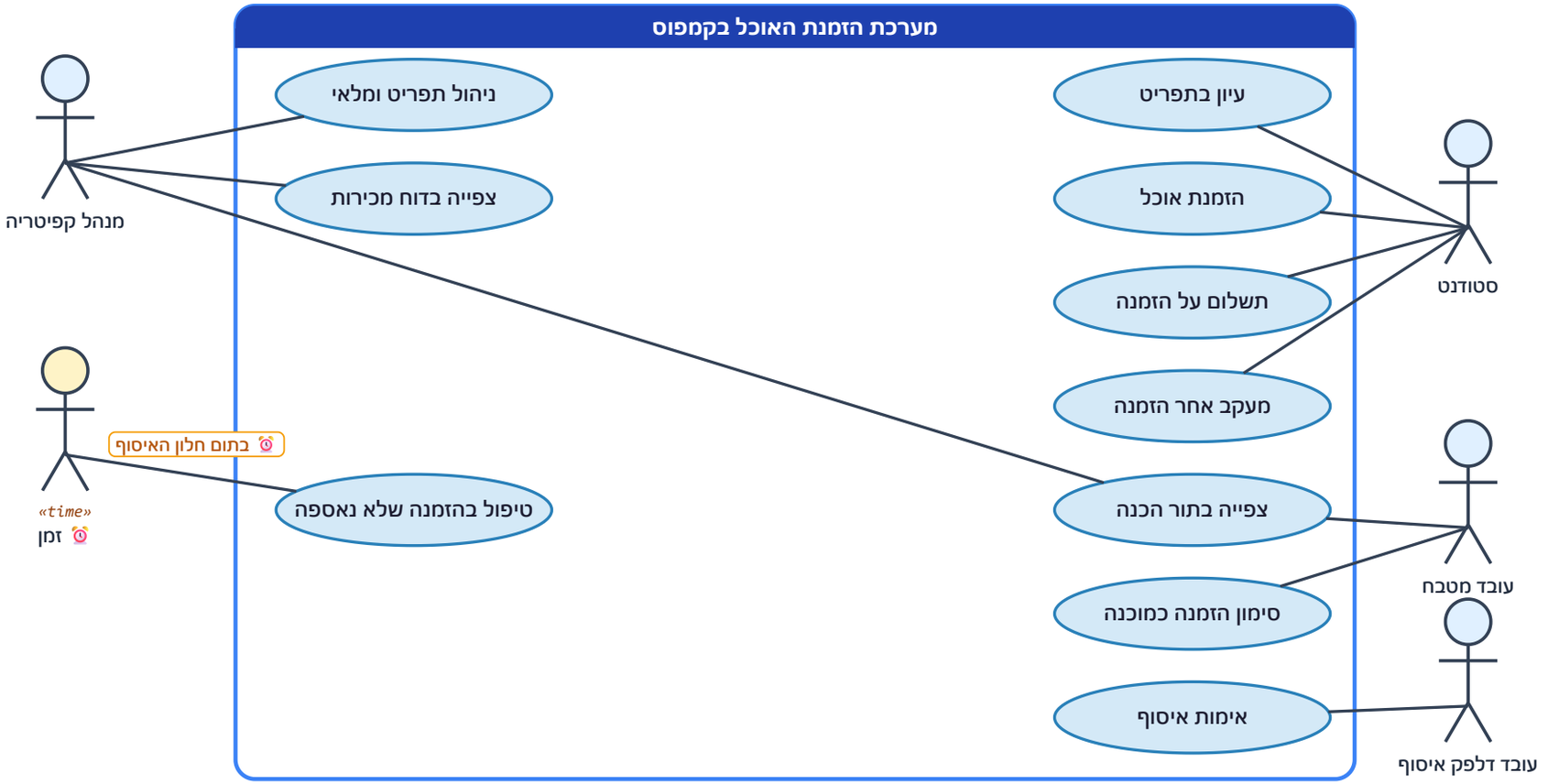
### 🤖 "פשוט אשאל את ה-AI?"

כן — אבל אתם מאמתים גם את רמת הפירוט וגם את הדוקטרינה (login זה לא תרחיש, extend במשורה...).

ניתוח דרישות

FAQ = Frequently Asked Questions · שאלות נפוצות · AI = Artificial Intelligence · בינה מלאכותית

# 1 / 5 התרשים של הקמפוס — שלב 1: הבסיס, מה כבר יש לנו



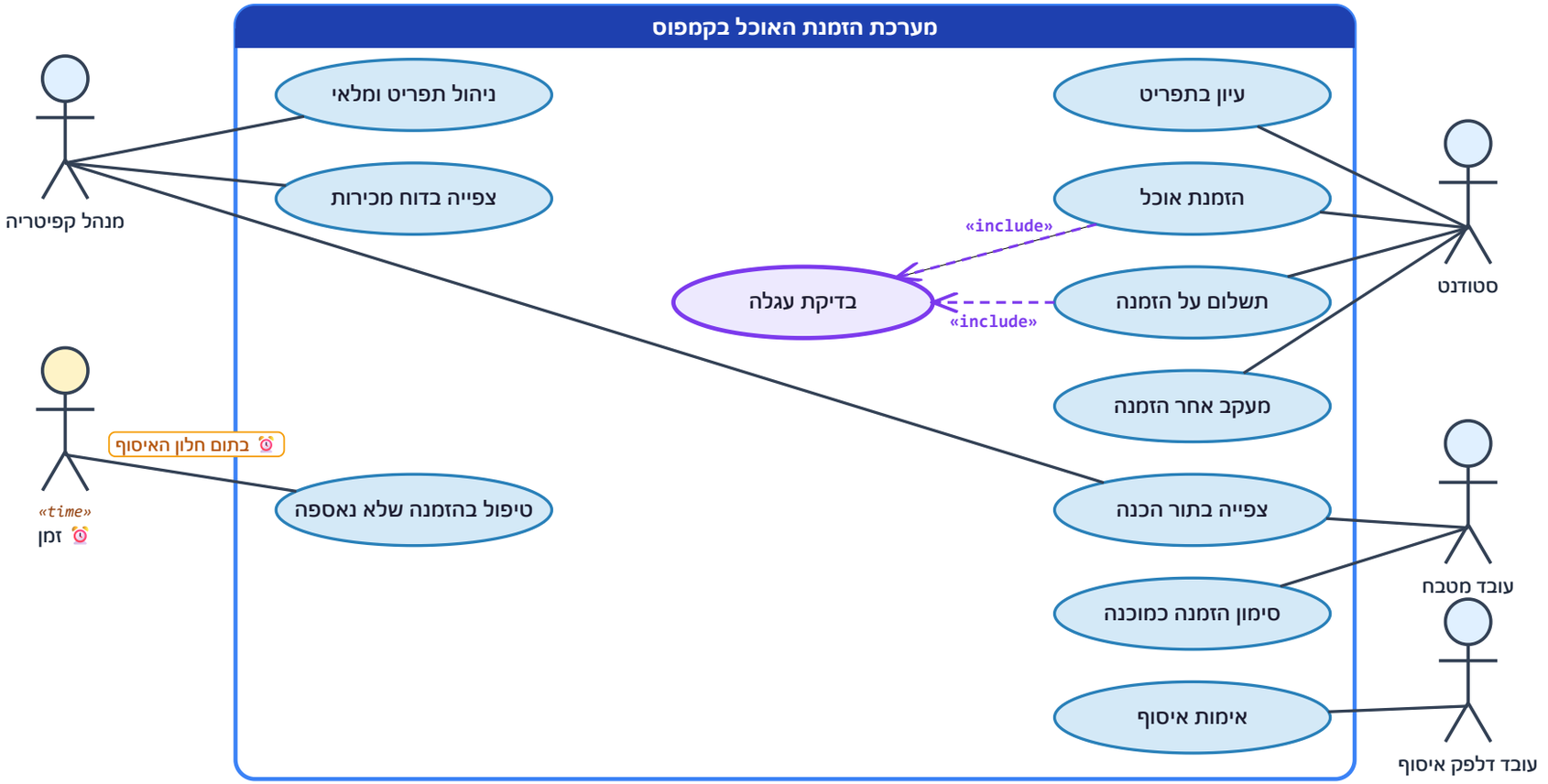

## נקודות לדיון

1. כל קו הוא קו של יוזם. שחקן = מי שיוזם. מי שרק מקבל הודעה — לא מצויר.
2. צפייה בתור הכנה — שני קווים: גם עובד המטבח וגם המנהל פותחים אותה בעצמם. שני יוזמים, לא "שחקן משני".
3. ⏰ זמן הוא שחקן: הוא מפעיל את הטיפול בהזמנה שלא נאספה — עם תווית תזמון חובה על הקו.
4. מה זז מאז שקף 45? הזמנה ותשלום פוצלו לשניים (משלמים על הזמנה קיימת), נוספו סימון כמוכנה ודוח מכירות — ומערכת הרישום ירדה: המודל המאומת מניח זיהוי בחשבון אוניברסיטאי.

**עשרה תרחישים, וכל אחד עם יוזם. עוד שניים ייוולדו מהקשרים: אחד שנקרא, אחד שמרחיב.**

ניתוח דרישות

# 2 / 5 שלב 2: «include» — בדיקת עגלה נקראת, לא יוזמים אותה



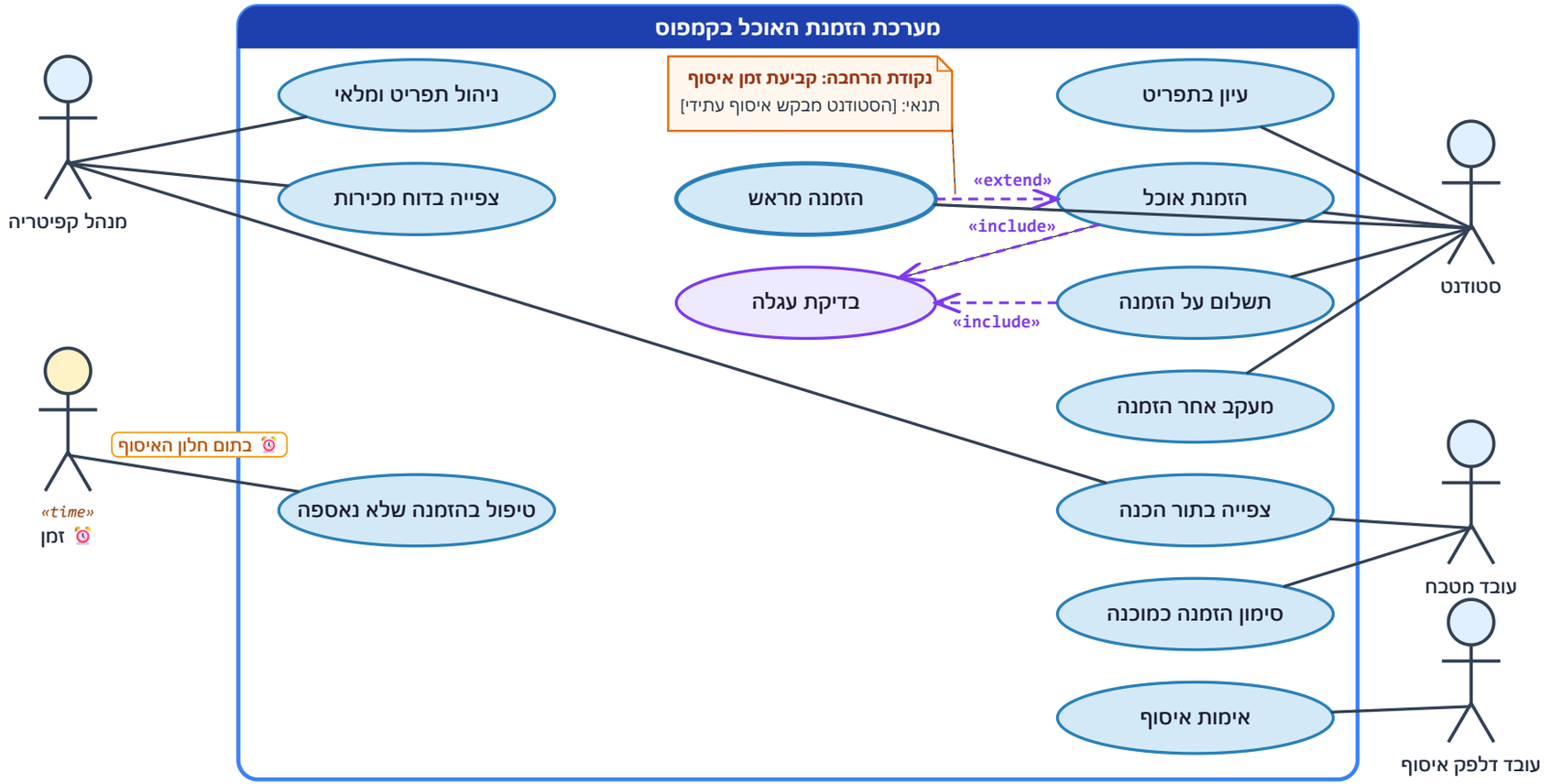

## נקודות לדיון

1. שני בסיסים מצביעים עליה — הזמנת אוכל וגם תשלום על הזמנה. «include» עם בסיס אחד? מקפלים פנימה כצעד.
2. אין לה שחקן. אף אחד לא יוזם בדיקת עגלה — קוראים לה. הזרימה שלה: שלושה צעדי מערכת, אפס צעדי שחקן.
3. רצה תמיד — בלי תנאי. אם זה קורה רק לפעמים, זה כבר לא include.
4. כיוון החץ: מהבסיס אל המשותף. חץ מקווקו, ראש פתוח, והמילה «include» עליו.

**«include» = שורה אחת בזרימה של כל בסיס: "2. «include» בדיקת עגלה".**

ניתוח דרישות

# 3 / 5 שלב 3: «extend» — הזמנה מראש מרחיבה את הזמנת אוכל



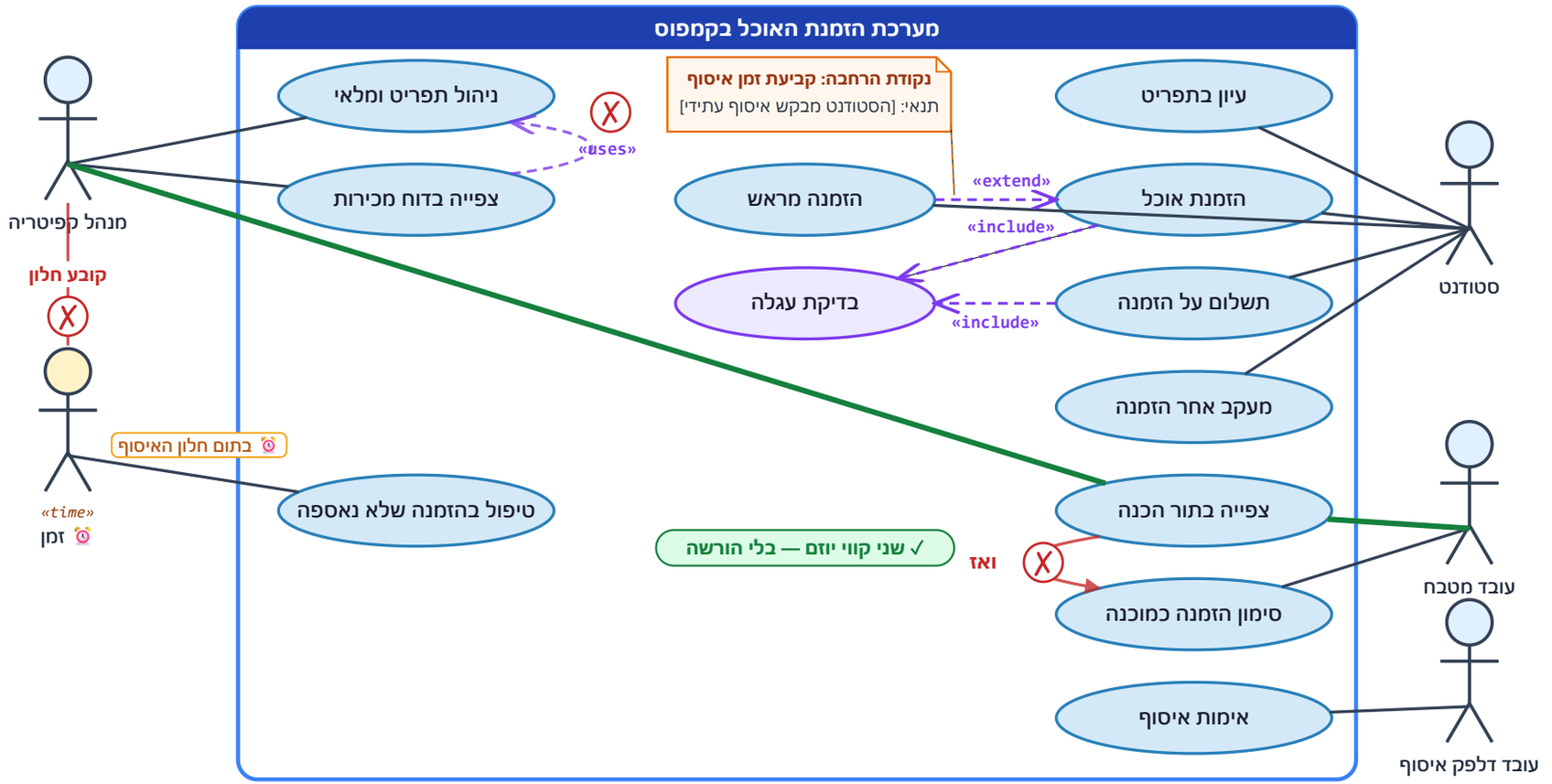

## נקודות לדיון

1. החץ הפוך: מהמרחיב (הזמנה מראש) אל הבסיס (הזמנת אוכל). הבסיס רץ מצוין בלעדיו.
2. הבסיס אומר איפה — נקודת הרחבה: קביעת זמן איסוף. ההרחבה אומרת מתי — [הסטודנט מבקש איסוף עתידי]. שניהם בפתק על החץ.
3. אף אחד לא קורא לה. ל-`include` יש שורה בזרימה; ל-`extend` אין — הוא נכנס בנקודה, רק אם התנאי מתקיים.
4. מותר לה שחקן: הסטודנט יוזם הזמנה מראש. (ליעד `include` — אסור.)
5. למה בתרשים ולא תרחיש נסתר? כי המנהל צריך לראות את זה — `no-show` הוא הדאגה שלו.

**include = תמיד, נקרא · extend = בתנאי, לא נקרא. שניהם חץ מקווקו — הכיוון והתנאי הם ההבדל.**

ניתוח דרישות

# 4 / 5 שלב 4: מה במתכוון לא ציירנו



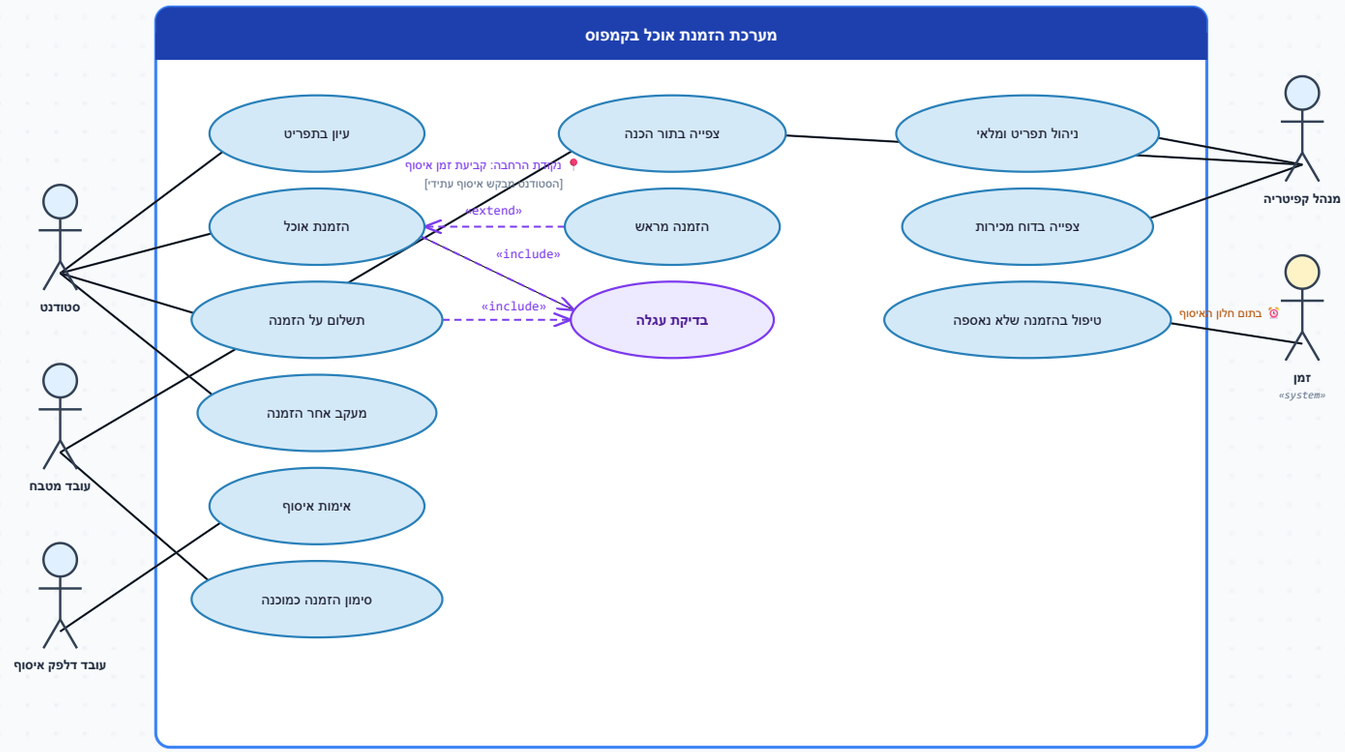

## נקודות לדיון

1. ✗ "ואז" — אין חצי רצף בין תרחישים. זה לא `flowchart`; הסדר חי ב-`flow of events`.
2. ✗ «uses» — מת עם UML 1. היום רק «include», ורק כשיש שני בסיסים. תלות בנתונים אינה קשר.
3. ✗ קו בין שחקנים — המנהל "קובע את חלון האיסוף"? אם המערכת עושה זאת, זה תרחיש; אם לא — זה מחוץ לגבול. בין שחקנים מותרת רק הורשה.
4. ✓ ובלי הורשה. צוות קפיטריה ▲ הוצג בשקף ההורשה כאפשרות, ושני התנאים מתקיימים — אז מותר. אבל שחקן נוסף בשביל תרחיש משותף אחד לא קונה כלום: שני קווי יוזם אומרים בדיוק את זה. KISS.

**ברירת המחדל: אפס «include», אפס «extend», אפס הורשה. כל סימן נוסף חייב לקנות משהו.**

ניתוח דרישות

# 5 / 5 שלב 5: התרשים המלא — חי




## ✓ רשימת בדיקה

- ✓ גבול — מה בפנים, מה בחוץ; שער התשלום איננו: קוראים לו, הוא לא יוזם
- ✓ לכל תרחיש יוזם — או שהוא נקרא («include») או מרחיב («extend»)
- → «include» — מהבסיס אל המשותף; שני בסיסים; בלי שחקן
- ← «extend» — מהמרחיב אל הבסיס, עם נקודת הרחבה + תנאי
- ⏰ תווית תזמון על הקו של שחקן לא-אנושי — "בתום חלון האיסוף"
- ✓ אין חצי רצף בין תרחישים, אין «uses», אין קו בין שחקנים
- ✓ KISS — 12 תרחישים, include אחד, extend אחד, אפס הורשה

**🖱️ התרשים חי**

לחצו על תרחיש לפתיחת ה-flow of events שלו — ועל קו, כדי לראות מי יוזם

ניתוח דרישות



KISS = Keep It Simple, Stupid · שמור על פשטות

# מניתוח לפיתוח — ותפקיד האנליסט

המפרט שבניתם עובר הלאה: ה-AI כותב את הקוד — ואתם מחליטים ומאתגרים

**1**
**תרחישים + תיעוד**
מה שבנינו היום

←

**2**
**CLAUDE.md**
המפרט שנמסר לפיתוח

←

**3**
**פיתוח**
ה-AI כותב קוד · הצוות: לא-פונקציונלי ושילוב

←

**הרצאה 13**
מהמפרט למערכת רצה

**💡 המפרט הוא הפרומפט**
ההבנה שלכם היא התקרה של מה ש-AI יפיק: מפרט חד ← קוד טוב; עמימות ← אותו באג, בביטחון ובקנה מידה. מודל תרחישי השימוש הוא בדיוק מפרט כזה.

**פעם**
רוב הזמן הלך על ייצור התוצרים — לצייר תרשימים, לכתוב תיעוד, לעצב טבלאות.

**היום**
הערך עובר להחלטה ולאימות: לשאול את השאלות הנכונות, להגדיר היקף, ולאתגר את מה ש-AI מייצר.

**שיקול דעת, ידע בתחום, ויכולת לאתגר — המיומנות הנדירה שנשארת.**

github.com/dcodish/SAD-sample-project — פרויקט הדוגמה להרצאה 13

ניתוח דרישות

# תרגול: מהמסמך לתרשים — ואז ביקורת

מזינים מסמך אפיון מלא ל-AI, מקבלים תרשים — ואתם השופטים

### מסמך אפיון מלא
ארגון · מצב קיים · ראיונות · 100+ דרישות

### ה-AI מייצר תרשים
שחקנים, תרחישים, קשרים — תוך שניות

### אתם מבקרים
מה חסר? מה הומצא? מה הונח?

### ספרייה עירונית 'אשכול' בכיתה
134 דרישות · FR · NFR · DD · BR · הורשת שחקנים · תזמון

רשת של 3 סניפים: השאלות, הזמנות, קנסות, רכש מספקים. נריץ יחד בכיתה ונפרק את התרשים על המסך.

באתר הקורס: materials · 3-ניתוח-דרישות · תרגול-בכיתה

### חניון 'כיכר המרכז' בבית
118 דרישות · FR · NFR · DD · BR · שער תשלום = API · תזמון

חניון עירוני בתשלום: כניסה, יציאה, מנויים, אכיפה. אותו תהליך בדיוק — זו מטלת הבית (השקף האחרון).

באתר הקורס: materials · 3-ניתוח-דרישות · תרגיל-בית

**⚠ המיומנות שמתרגלים**

השאלה איננה "מה ה-AI ייצר" אלא "מה הוא פספס, המציא, או הניח" — שחקן חסר, «include» מומצא, מערכת חיצונית שסומנה כשחקן במקום קריאת API, גבול שגוי.

**אותו מסמך, אותו AI — ושתי כיתות יקבלו ביקורת שונה. הביקורת היא התוצר.**

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית · AI = Artificial Intelligence · בינה מלאכותית

ניתוח דרישות

# סיכום וסגירה

מה למדנו — ולאן ממשיכים

- UML רלוונטי — ודווקא בעידן AI הסמנטיקה חשובה יותר
- Use Case = שחקנים + תרחישים + גבול מערכת
- ארבעה קשרים: association, include, extend, generalization
- התיעוד (Flow of Events) הוא התוכן; התרשים הוא המפה
- AI מאיץ כל שלב — ואתם מאתגרים ומאמתים

**הבא בתור: תרשים מחלקות (Class Diagram) — הרצאה 6-7**

נעבור מ'מה המערכת עושה' ל'ממה היא בנויה'.

# מטלת בית 🏠

אותו תהליך שעשינו בכיתה — על הדוגמה שלא הרצנו

**קחו את מסמך האפיון של החניון 'כיכר המרכז'**

מסמך מלא: תיאור הארגון, מצב קיים, ראיונות, ו-118 דרישות מפורטות (FR/NFR/DD/BR). הקובץ באתר הקורס: `3 · materials-ניתוח-דרישות · תרגיל-בית · מסמך-אפיון-חניון`

**1**
**הזינו ל-AI**
תנו את המסמך ל-Claude ובקשו תרשים flow of + use case events לשניים-שלושה תרחישים מרכזיים.

**2**
**בקרו לפי הדוקטרינה**
מי באמת שחקן ומי API שאנחנו קוראים לו? יש «include» מומצא (למשל login)? «extend» שהוא בעצם חריג? גבול נכון?

**3**
**תקנו ונמקו**
הציעו תרשים מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון שלכם נכון.

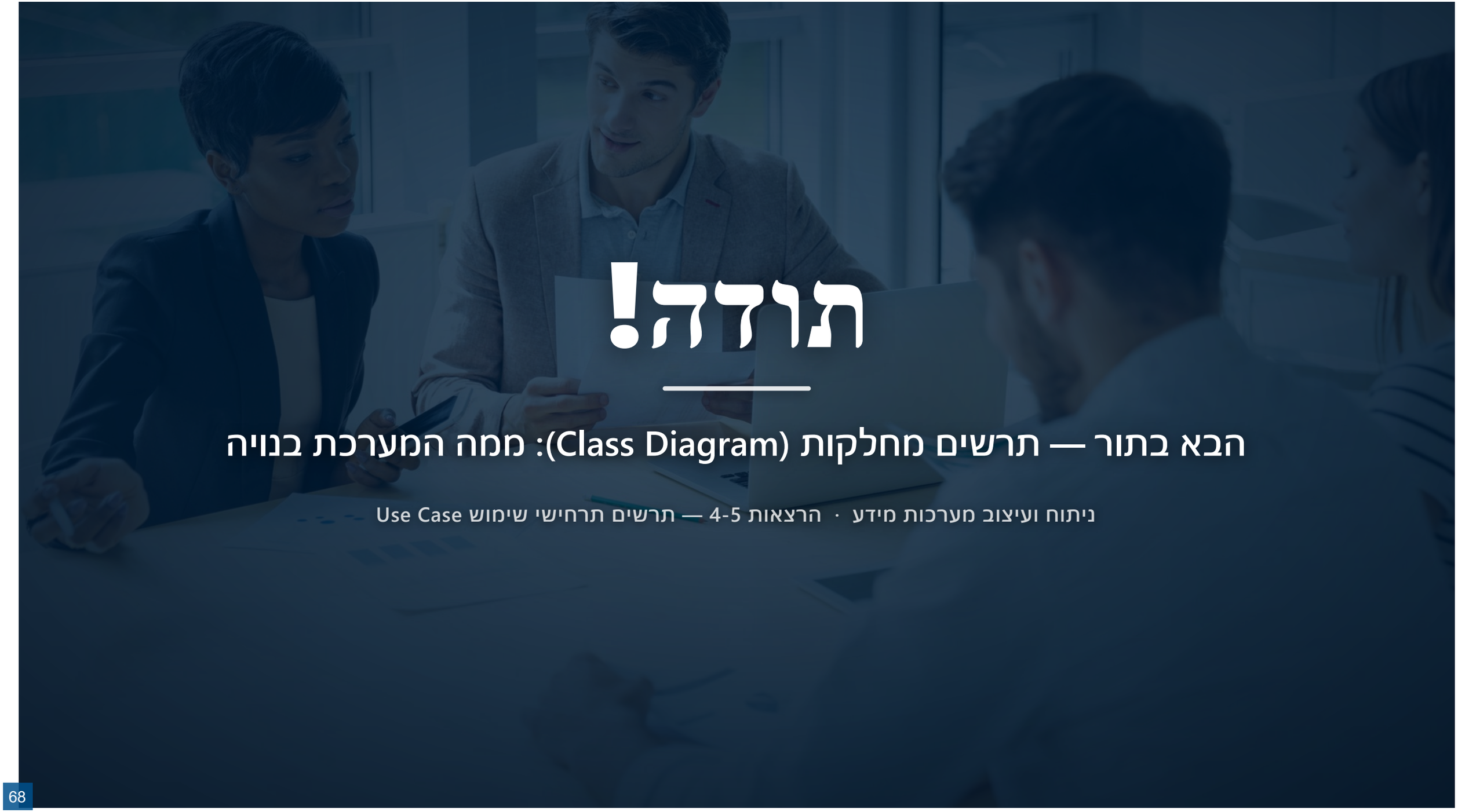
**📤 מה מגישים**

- הפלט המקורי של ה-AI (התרשים והתיעוד שקיבלתם) + הפרומפט ששלחתם.
- טבלת ביקורת: לכל ליקוי — מה הבעיה, לאיזה כלל היא נוגעת, והתיקון.
- התרשים המתוקן שלכם (מהכלי או ביד), ופסקה קצרה: מה ה-AI דווקא עשה טוב.

**לא מדרגים כמה יפה ה-AI צייר — מדרגים כמה חד הייתה הביקורת שלכם.**

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

ניתוח דרישות



# תודה!

## הבא בתור — תרשים מחלקות (Class Diagram): ממה המערכת בנויה

ניתוח ועיצוב מערכות מידע · הרצאות 4-5 — תרשים תרחישי שימוש Use Case